

MEMORIA DE ESTADÍA

PARA OBTENER EL TÍTULO EN

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

PLATAFORMA DIGITAL PARA LA GESTIÓN Y CONSULTA DE
RECURSOS BIBLIOGRÁFICOS DEL INSTITUTO DE FORMACIÓN Y
CAPACITACIÓN PROFESIONAL DE LA FISCALÍA GENERAL DEL
ESTADO DE OAXACA

FISCALÍA GENERAL DEL ESTADO DE OAXACA

PRESENTA

ORTIZ MARTÍNEZ VÍCTOR HUGO

“ CONOCIMIENTO PRÁCTICO QUE TRANSFORMA ”

SAN PABLO HUIXTEPEC OAXACA A JUEVES 10 DE ABRIL DEL 2025

MEMORIA DE ESTADÍA

PARA OBTENER EL TÍTULO EN

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

FISCALÍA GENERAL DEL ESTADO DE OAXACA

PRESENTA

ORTIZ MARTÍNEZ VÍCTOR HUGO

ASESOR ACADÉMICO

Mtro. Elmer Leonardo Beltrán Jiménez

ASESOR EMPRESARIAL

Lic. Nancy Nayeli Vázquez Agustín

“ CONOCIMIENTO PRÁCTICO QUE TRANSFORMA ”

SAN PABLO HUIXTEPEC OAXACA A JUEVES 10 DE ABRIL DEL 2025



OAXACA
GOBIERNO DEL ESTADO



2025
"Bicentenario de la Primera Constitución Política
del Estado Libre y Soberano de Oaxaca"

San Pablo Huixtepec, Oaxaca, a 10 de abril de 2025.

DIRECCIÓN DE TECNOLOGÍAS DE LA INFORMACIÓN
OFICIO.UTVCO.TI.80.2025
ASUNTO: DICTAMEN

ORTIZ MARTINEZ VICTOR HUGO
ESTUDIANTE DE LA CARRERA DE
DESARROLLO Y GESTIÓN DE SOFTWARE
P R E S E N T E

Con la finalidad de iniciar su trámite de titulación para obtener el grado de "Ingeniero" en Desarrollo y Gestión de Software y después de haber concluido satisfactoriamente su periodo de estadía en la empresa: "FISCALÍA GENERAL DEL ESTADO DE OAXACA", donde desarrolló el proyecto:

"PLATAFORMA DIGITAL PARA LA GESTIÓN Y CONSULTA DE RECURSOS
BIBLIOGRÁFICOS DEL INSTITUTO DE FORMACIÓN Y CAPACITACIÓN
PROFESIONAL DE LA FISCALÍA GENERAL DEL ESTADO DE OAXACA"

Esta dirección autoriza la impresión de la memoria respectiva.

ATENTAMENTE
"Conocimiento práctico que transforma"

MTRO. MIGUEL MÚÑOZ CRUZ
ENCARGADO DE LA DIVISIÓN DE CARRERA DE TECNOLOGÍAS
DE LA INFORMACIÓN

c.c.p. Expediente.
MMC/aacr



OAXACA
GOBIERNO DEL ESTADO



SEP
SECRETARÍA DE
EDUCACIÓN PÚBLICA



Av. Universidad s/n, San Pablo Huixtepec. Zim. Oax. C. P. 71265



UT Valles Centrales de Oaxaca



@UTVCO

RECONOCIMIENTOS

El desarrollo de este proyecto no habría sido posible sin el apoyo y respaldo de diversas instituciones y personas que brindaron su orientación, recursos y confianza a lo largo de mi formación profesional.

En primer lugar, extendo mi más sincero agradecimiento a la Fiscalía General del Estado de Oaxaca (FGEO) y, de manera particular, al Instituto de Formación y Capacitación Profesional (IFCP), por abrirme las puertas y ofrecer un entorno profesional donde pude aplicar y consolidar los conocimientos adquiridos durante mi trayectoria universitaria.

De igual forma, agradezco profundamente a la Universidad Tecnológica de los Valles Centrales de Oaxaca (UTVCO), institución que ha sido clave en mi formación académica, no solo por brindarme educación, sino también por fomentar la innovación, la disciplina y el compromiso ético en el quehacer profesional.

Un reconocimiento especial al Mtro. Elmer Leonardo Beltrán Jiménez, asesor académico, por su guía honesta, experiencia, retroalimentación constante y por confiar en mis capacidades para el desarrollo e implementación de este proyecto. Así como a la Lic. Nancy Nayeli Vázquez Agustín, asesora académica, por su acompañamiento, apoyo incondicional y dedicación durante este proceso formativo.

Finalmente, pero no menos importante, quiero expresar mi más profundo agradecimiento a mis padres, quienes han sido el pilar fundamental de mi vida. Gracias por su amor, sacrificio y apoyo incondicional en cada paso de este camino.

A todos ustedes, mi más sincera gratitud.

RESUMEN

Este proyecto consistió en el diseño, desarrollo e implementación de una plataforma web para la gestión de una biblioteca digital interna para el Instituto de Formación y Capacitación Profesional (IFCP). La solución se desarrolló con tecnologías modernas como Angular v19 para el frontend y NestJS como backend, integrados a una base de datos MariaDB. La justificación principal se basó en la necesidad de digitalizar el acceso, consulta y administración de materiales bibliográficos, eliminando barreras físicas y mejorando la experiencia del usuario.

El sistema se planteó con roles de usuario diferenciados (admin, partner y user), permitiendo así funcionalidades controladas según el perfil, como la administración de libros, categorías y usuarios. Entre los objetivos clave se encontraron: automatizar procesos manuales, facilitar el acceso remoto a documentos en formato PDF, y permitir la gestión segura de usuarios. Se siguió una metodología basada en el enfoque ágil, con iteraciones funcionales y revisión continua mediante control de versiones con Git y GitHub. Entre los resultados, se logró una interfaz intuitiva adaptada al diseño propuesto en Figma, implementación de funcionalidades completas como carga de libros con miniatura generada automáticamente, clasificación por categorías, gestión de permisos, y visualización personalizada por rol.

Además, se documentó el sistema bajo arquitectura modular feature-based, contemplando su futura escalabilidad y despliegue en un servidor productivo. Finalmente, se concluye que el desarrollo cumple satisfactoriamente con los objetivos planteados, brindando una herramienta robusta, accesible y administrable, lista para ser desplegada y utilizada institucionalmente.

INDICE

INDICE	6
ÍNDICE DE FIGURAS	8
ÍNDICE DE TABLAS	10
ÍNDICE DE ANEXOS	10
I. INTRODUCCIÓN	11
1.1 Descripción De La Empresa	12
1.1.1 Datos generales	12
1.1.2 Misión	13
1.1.3 Visión	13
1.2 Antecedentes	13
II. DESCRIPCIÓN DEL PROYECTO	15
2.1 Planteamiento del Problema	15
2.2 Justificación	17
2.3 Objetivo General y Específicos	18
2.3.1 Objetivo General	18
2.3.2 Objetivos Específicos	18
2.4 Alcances y limitaciones	19
2.4.1 Alcances	19
2.4.2 Limitaciones	20
III. HIPÓTESIS	23
IV. DESARROLLO DEL PROYECTO	24
4.1 Marco teórico	24
4.1.1 Desarrollo de software	24
4.1.2 Arquitectura de tres capas	25
4.1.3 Bases de datos	32
4.1.4 Seguridad informática	35
4.1.5 Diseño centrado en el usuario (UX)	37
4.1.6 Metodología de trabajo en el desarrollo de software	40
4.1.7 Metodologías ágiles	41
4.1.8 Pruebas unitarias	41

V.	METODOLOGÍA.....	42
5.1	Requerimientos Funcionales	42
5.1.17	Cronograma de actividades	45
5.2	Base de Datos	52
5.3	Estilos e Historia de Usuario	56
5.4	Guía de Estilos.....	59
VI.	RESULTADOS	87
VII.	CONCLUSIONES	89
	BIBLIOGRAFÍA.....	90
	ANEXOS	93

ÍNDICE DE FIGURAS

FIGURA 1 MACRO LOCALIZACIÓN.....	12
FIGURA 2 MICRO LOCALIZACIÓN.....	12
FIGURA 3 LOGOTIPO DEL INSTITUTO DE FORMACIÓN Y CAPACITACIÓN PROFESIONAL.....	14
FIGURA 4 LOGO DE ANGULAR.....	26
FIGURA 5 LOGO DE NESTJS.....	27
FIGURA 6 LOGO DE TYPESCRIPT.....	28
FIGURA 7 LOGO DE JAVA SCRIPT.....	28
FIGURA 8 LOGO DE NODEJS.....	29
FIGURA 9 LOGO DE NPM.....	29
FIGURA 10 LOGO DE VISUAL STUDIO CODE.....	30
FIGURA 11 LOGO DE GIT.....	31
FIGURA 12 LOGO DE GITHUB.....	31
FIGURA 13 LOGO DE MARIADB.....	34
FIGURA 14 LOGO DE HEIDISQL.....	34
FIGURA 15 LOGO DE POSTMAN.....	35
FIGURA 16 ANGULAR MATERIAL LOGO.....	37
FIGURA 17 LOGO TAILWIND CSS.....	38
FIGURA 18 LOGO CSS.....	38
FIGURA 19 LOGO HTML.....	39
FIGURA 20 LOGO DE FIGMA.....	40
FIGURA 21 TABLERO SCRUM EN LA APLICACIÓN JIRA.....	50
FIGURA 22 TABLAS DE LA BASE DE DATOS.....	52
FIGURA 23 CÓDIGO CREATE PARA LA TABLA 'BOOKS'.....	52
FIGURA 24 CÓDIGO CREATE PAR LA TABLA 'CATEGORIES'.....	53
FIGURA 25 CÓDIGO CREATE PARA LA TABLA 'SAVED_BOOK'.....	54
FIGURA 26 CÓDIGO CREATE PARA LA TABLA 'USERS'.....	55
FIGURA 27 PALETA DE COLORES.....	60
FIGURA 28 FUENTE POPPINS.....	61
FIGURA 29 OPTIONS REGULAR.....	62
FIGURA 30 TITLE.....	62
FIGURA 31 TITLE2 MEDIUM.....	62
FIGURA 32 TEXT REGULAR.....	63
FIGURA 33 INFO LIGHT.....	63
FIGURA 34 TITLE_BOOK_PREVIEW EXTRABOLD.....	63
FIGURA 35 INFO_BOOK_PREVIEW.....	64
FIGURA 36 BUTTONS MEDIUM.....	64
FIGURA 37 INPUTS DE LOGIN Y REGISTER.....	64
FIGURA 38 BOTONES DE REGISTRO, INICIO DE SESIÓN, CATEGORÍA, ELIMINAR Y EDITAR.....	65
FIGURA 39 ÍCONOS Y LOGOTIPOS DE LA FGEO Y DEL IFCP.....	66
FIGURA 40 SIDEBARDS DE ROL USER Y ADMIN.....	68
FIGURA 41 HEADER GENERAL.....	69
FIGURA 42 MODAL DE PREVISUALIZACIÓN DE LIBROS.....	70
FIGURA 43 WELCOME.....	72
FIGURA 44 LOGIN.....	73
FIGURA 45 REGISTER.....	74
FIGURA 46 HOME-ADMIN.....	75
FIGURA 47 HOME-USER.....	76
FIGURA 48 CATEGORIES.....	77

FIGURA 49 ESTRUCTURA DE CARPETAS DE MANERA VISUAL.	79
FIGURA 50 ARCHIVO .ENV.....	81
FIGURA 51 ARCHIVO ENVIROMENT.TS.....	81
FIGURA 52 REPOSITORIO EN GITHUB FRONTEND	84
FIGURA 53 REPOSITORIO EN GITHUB BACKEND.	84
FIGURA 54 COMMITS TOTALES EN FRONTEND.....	85
FIGURA 55 COMMITS TOTALES EN BACKEND.	86

ÍNDICE DE TABLAS

TABLA 1 CRONOGRAMA DE ACTIVIDADES	45
TABLA 2 TABLA BOOKS.....	53
TABLA 3 CATEGORIES.....	54
TABLA 4 SAVED_BOOK	54
TABLA 5 USERS	55

ÍNDICE DE ANEXOS

ANEXOS 1 BOCETO DE LA CREACIÓN DE TABLAS.....	93
ANEXOS 2 BOCETO DE LA FUNCIÓN SAVED_BOOK POR ID DE USUARIO.....	93
ANEXOS 3 BOCETO DE LA FUNCIÓN UPLOADS_BOOKS.	93

I. INTRODUCCIÓN

La presente memoria tiene como propósito brindar una perspectiva integral sobre el desarrollo de una plataforma web destinada a la gestión y consulta de recursos bibliográficos digitales para el Instituto de Formación y Capacitación Profesional (IFCP) de la Fiscalía General del Estado de Oaxaca. Esta iniciativa surge como una respuesta estratégica ante la necesidad institucional de contar con un sistema que centralice, organice y facilite el acceso a libros en formato PDF, fortaleciendo así los procesos de capacitación y actualización del personal. En esta sección se presenta al lector una visión clara del objetivo principal del trabajo, las razones que justificaron su desarrollo, los retos técnicos y funcionales enfrentados, las soluciones adoptadas, y un esbozo general de los contenidos que estructuran este documento.

La elección de este tema se fundamenta en la identificación de una problemática significativa dentro de la institución: la falta de una plataforma centralizada que permita a los empleados acceder de manera organizada y segura a recursos bibliográficos digitales. En un contexto donde la digitalización de contenidos se ha convertido en un estándar educativo y empresarial, la ausencia de una herramienta de este tipo limita el acceso y aprovechamiento de materiales educativos, afectando directamente la calidad de la formación profesional y la difusión del conocimiento. Este proyecto busca no solo resolver esta carencia, sino también posicionar a la institución como un modelo innovador en la gestión de recursos educativos y culturales.

1.1.2 Misión

Formar y profesionalizar a mujeres y hombres con ética, vocación de servicio y respeto a los derechos humanos, desarrollando habilidades y competencias bajo estándares nacionales e internacionales en beneficio de la sociedad.

1.1.3 Visión

Consolidarse como una institución líder e innovadora a nivel nacional, con un modelo de formación vanguardista que fortalezca el sistema de justicia penal y eleve el nivel de profesionalización en la procuración de justicia.

1.2 Antecedentes.

El Instituto de Formación y Capacitación Profesional (IFCP) es una entidad adscrita a la fiscalía general del Estado de Oaxaca, ubicada en el Centro Administrativo del Poder Ejecutivo y Judicial General Porfirio Díaz, Edificio Jesús "Chu" Rasgado y Álvaro Carrillo, en Reyes Mantecón, San Bartolo Coyotepec, Oaxaca.

El IFCP ofrece programas educativos teórico-prácticos dirigidos a agentes estatales de investigación, peritos, analistas, facilitadores y ministerios públicos, con el objetivo de fortalecer el Sistema de Justicia Penal y de Seguridad Pública Estatal. Además, busca generar estrategias académicas a través de la vinculación nacional e internacional.

Fue creada para responder a los desafíos del Sistema Penal Acusatorio, implementado en México desde 2008 y adoptado formalmente en Oaxaca hacia 2016. Su origen se vincula a la necesidad de profesionalizar a los operadores del sistema de justicia (agentes de investigación, ministerios públicos, peritos y personal administrativo) mediante capacitación especializada en técnicas modernas de procuración de justicia, derechos humanos y perspectiva de género.

El IFCP surgió entre 2015 y 2017, durante la consolidación del nuevo modelo de justicia penal en Oaxaca. Su establecimiento se formalizó bajo acuerdos internos de la Fiscalía, respaldados por la Ley Orgánica de la institución y alineados con el Plan Estatal de Desarrollo 2016-2022, que priorizó la modernización del sector seguridad y justicia.

Inició operaciones en el Centro Administrativo del Poder Ejecutivo y Judicial General Porfirio Díaz, ubicado en Reyes Mantecón, San Bartolo Coyotepec. Este espacio estratégico refleja su integración en la estructura gubernamental y su enfoque en la coordinación interinstitucional.

Figura 13 Logotipo del Instituto de Formación Y Capacitación Profesional



Referencia. Obtenido <https://fge.oaxaca.gob.mx/>

II. DESCRIPCIÓN DEL PROYECTO

2.1 Planteamiento del Problema

En un contexto actual en el IFPC de la FGEO, el acceso a recursos digitales, como libros electrónicos, es fundamental para fomentar la investigación, el aprendizaje y la preservación del conocimiento; sin embargo, en el área de trabajo, se ha identificado una problemática significativa: la ausencia de una plataforma centralizada que permita a los trabajadores de la fiscalía general del estado de Oaxaca acceder de forma organizada y segura a libros en formato digital. Esta situación dificulta el acceso y aprovechamiento de los recursos y materiales educativos, especialmente en un contexto donde el uso de herramientas digitales en la actualidad se ha vuelto imprescindible.

En particular, se observa una desconexión entre las herramientas de aprendizaje disponibles y los recursos, lo cual limita el alcance de la educación digital y la difusión de materiales propios de la. Actualmente, no existe un sistema eficiente que permita gestionar libros por categorías, autores o editoriales; ni que facilite su descarga en formato PDF (que sería el formato principal para los libros) con un control adecuado. Esto no solo afecta la accesibilidad, sino que también repercute en la posibilidad de preservar y compartir el conocimiento generado.

Los cursos impartidos por la Fiscalía abarcan diversas áreas, como derecho, investigación, tecnología y gestión pública, entre otras. Para que estos cursos sean realmente efectivos, es fundamental que los participantes cuenten con materiales de apoyo actualizados y especializados, como libros, manuales y guías en formato digital.

Además, la falta de seguridad en plataformas similares podría derivar en riesgos como el acceso no autorizado, la manipulación de archivos o incluso la distribución de contenido malicioso.

Por lo tanto, resulta más beneficioso para el Instituto de Formación y Capacitación Profesional desarrollar una biblioteca digital que permita la gestión organizada de libros; y garantice la seguridad, tanto en el acceso como en la descarga de los materiales.

Debido los temas anteriores se ve conveniente construir una solución tecnológica que no solo optimice la gestión y distribución de recursos educativos digitales, sino que también promueva el uso controlado, responsable y seguro de los mismos; al tiempo que fomente la conservación de la información y patrimonio bibliográfico.

2.2 Justificación

La implementación de una biblioteca digital para los empleados en la fiscalía y en el Instituto de Formación y Capacitación Profesional, responde a la necesidad de promover el acceso eficiente, seguro y organizado a los recursos bibliográficos en formato digital.

En un entorno donde la digitalización de contenidos se ha convertido en un estándar educativo y cultural, esta solución permitirá preservar, difundir y aprovechar la información relacionada a temas específicos del área, contribuyendo al desarrollo académico y profesional de los usuarios.

Esto radica en la relevancia de centralizar los recursos bibliográficos locales en una plataforma accesible y funcional, que facilite la consulta y descarga de materiales en formato PDF. Esto no solo mejorará la experiencia del usuario al permitir búsquedas por categorías, autores y editoriales, sino que también fomentará la conservación del patrimonio cultural y educativo, al digitalizar y proteger información valiosa para la comunidad.

La inclusión de medidas de seguridad robustas garantizará que los materiales estén protegidos contra accesos no autorizados o alteraciones maliciosas, generando confianza tanto en los administradores como en los usuarios finales.

Por consecuencia, el proyecto responde a la demanda de una herramienta digital que promueva el aprendizaje y la accesibilidad al conocimiento en todos los niveles.

Al desarrollar esta biblioteca digital, se busca no solo atender una necesidad específica de la institución, sino también posicionarla como un modelo innovador y sostenible en la gestión de recursos educativos y culturales.

2.3 Objetivo General y Específicos

2.3.1 Objetivo General

Diseñar y desarrollar una plataforma de gestión y consulta de recursos bibliográficos en el Instituto de Formación y Capacitación Profesional de la FGEO (Fiscalía General del Estado de Oaxaca) para facilitar el acceso, organización y distribución de libros digitales.

2.3.2 Objetivos Específicos

- Obtener requerimientos del sistema
- Definir las plataformas o programas para el diseño del sistema
- Elegir las tecnologías con la que se va desarrollar el sistema
- Definir guía de estilos
- Diseñar la base de datos (relacional)
- Diseñar e implementar un sistema de carga y descarga de libros en formato PDF, permitiendo a los usuarios visualizar y obtener los materiales de manera rápida y eficiente, sin comprometer la seguridad de los archivos.
- Desarrollar una base de datos relacional en MariaDB que almacene y organice la información de los libros, autores, categorías y editoriales, asegurando la correcta estructuración y gestión de los datos bibliográficos.
- Implementar un sistema de autenticación que permita a los usuarios registrados acceder a la biblioteca digital.
- Garantizar la seguridad de la información y evitar vulnerabilidades, mediante la implementación de prácticas de codificación segura.
- Optimizar el rendimiento de la plataforma para garantizar tiempos de respuesta adecuados en la consulta y descarga de materiales.
- Implementar las pruebas correspondiente al sistema para descartar fallos u otros aspectos.
- Desplegar la plataforma en un entorno de servidor local.

2.4 Alcances y limitaciones

2.4.1 Alcances

El proyecto de desarrollo de una plataforma de gestión y consulta de recursos bibliográficos para el Instituto de Formación y Capacitación Profesional de la Fiscalía General del Estado de Oaxaca será la creación de una solución tecnológica integral que permita centralizar, organizar y facilitar el acceso a libros digitales en formato PDF. Esta plataforma estará diseñada para satisfacer las necesidades específicas de los empleados y usuarios internos, brindando una herramienta eficiente y segura para la consulta, descarga y gestión de materiales bibliográficos.

Dicho proyecto englobará desde la obtención de los requerimientos del sistema hasta el despliegue de la plataforma en un entorno de servidor local. En primer lugar, se realizará un análisis detallado de las necesidades de los usuarios y los recursos bibliográficos disponibles, con el fin de definir las funcionalidades clave del sistema.

Posteriormente, se procederá al diseño y desarrollo del back-end, donde se implementará una API RESTful utilizando NestJS. Esta API será responsable de gestionar la lógica de negocio, incluyendo la carga, consulta y descarga de libros, así como la interacción con la base de datos relacional en MariaDB.

En cuanto al front-end, se desarrollará una interfaz de usuario intuitiva y responsive utilizando Angular. Esta interfaz permitirá a los usuarios realizar búsquedas de libros por categorías, autores o editoriales, visualizar detalles de los materiales y descargarlos en formato PDF de manera rápida y eficiente.

Además, se implementará un sistema de autenticación que garantice que solo los usuarios registrados puedan acceder a los recursos, asegurando la privacidad y seguridad de la información.

La base de datos relacional, diseñada en MariaDB, será un componente fundamental del sistema. En ella se almacenará y organizará la información de los libros, autores, categorías y editoriales, asegurando una estructuración adecuada de los datos bibliográficos.

Se establecerán relaciones entre estas entidades para facilitar la consulta y el filtrado de los materiales, optimizando así la experiencia del usuario.

En términos de seguridad, el proyecto incluirá la implementación de medidas robustas para proteger la integridad de los datos y prevenir vulnerabilidades comunes, como inyecciones SQL o accesos no autorizados. Se seguirán prácticas de codificación segura y se realizarán pruebas exhaustivas para garantizar que la plataforma cumpla con los estándares de seguridad requeridos.

Una vez desarrollada la plataforma, se llevarán a cabo pruebas de funcionalidad, rendimiento y seguridad para identificar y corregir posibles fallos. Finalmente, el sistema se desplegará en un entorno de servidor local, asegurando su disponibilidad para los usuarios internos de la Fiscalía.

2.4.2 Limitaciones

El desarrollo del proyecto de la biblioteca digital para el Instituto de Formación y Capacitación Profesional de la Fiscalía General del Estado de Oaxaca presenta una serie de limitaciones que es importante considerar para garantizar su éxito y sostenibilidad a largo plazo. Estas limitaciones abarcan aspectos técnicos, operativos y de recursos, y deben ser gestionadas adecuadamente para minimizar su impacto en la funcionalidad y utilidad de la plataforma.

En primer lugar, la accesibilidad restringida del sistema representa una limitación significativa. La plataforma estará diseñada exclusivamente para los empleados de la Fiscalía, lo que limita su alcance a un público específico y excluye a usuarios externos que podrían beneficiarse de los recursos bibliográficos disponibles. Esta restricción responde a la necesidad de proteger la información y garantizar el uso adecuado de los materiales, pero también reduce el potencial impacto social y educativo que la biblioteca digital podría tener fuera de la institución.

Otra limitación importante es la disponibilidad de recursos digitales. La utilidad de la plataforma dependerá en gran medida de la cantidad y calidad de los libros que sean digitalizados o adquiridos en formato PDF.

Si los recursos bibliográficos son limitados, desactualizados o no cubren las necesidades específicas de los usuarios, la plataforma podría no cumplir con su objetivo principal de fomentar el aprendizaje y la investigación. Además, el proceso de digitalización de libros físicos puede ser costoso y requerir tiempo, lo que podría retrasar la disponibilidad de los materiales.

La infraestructura tecnológica también representa un desafío. El sistema se ejecutará en un servidor local con Debian, lo que implica que su disponibilidad y rendimiento estarán condicionados por la capacidad de la infraestructura de la institución. Cualquier falla en el servidor, ya sea por problemas de hardware o software, podría afectar el acceso a la plataforma y generar interrupciones en el servicio. Asimismo, la capacidad de almacenamiento del servidor podría verse limitada a medida que se agreguen más libros en formato PDF, lo que requeriría una ampliación de la infraestructura en el futuro.

La capacitación del personal es otro aspecto crítico. Para garantizar el uso y administración correcta de la plataforma, será necesario capacitar tanto a la persona encargada de gestionar los libros como a los usuarios que accedan a dichos recursos.

La falta de capacitación adecuada podría generar problemas en la administración de contenidos, como errores en la carga de libros o dificultades para realizar búsquedas eficientes. Además, los usuarios podrían no aprovechar al máximo las funcionalidades de la plataforma si no están familiarizados con su uso.

En cuanto a la seguridad de la información, aunque se implementarán medidas robustas para proteger los datos y evitar accesos no autorizados, siempre existirá el riesgo de vulnerabilidades que puedan comprometer la integridad de los archivos y la privacidad de los usuarios.

El tiempo de desarrollo es un tema crucial en este caso. El proyecto tiene un plazo definido desde el 13 de enero hasta el 4 de abril, lo que implica que cualquier retraso en el desarrollo podría limitar la implementación de algunas funcionalidades o requerir ajustes en el alcance del proyecto. Esto podría afectar la calidad final de la plataforma y su capacidad para cumplir con todas las expectativas de los usuarios.

III. HIPÓTESIS

Si se desarrolla e implementa una biblioteca digital centralizada para los empleados del Instituto de Formación y Capacitación Profesional de la fiscalía general del estado de Oaxaca, entonces se optimizará la gestión y distribución de los recursos bibliográficos, mejorando significativamente la accesibilidad y la preservación del conocimiento local.

La creación de una plataforma digital que organice libros por categorías, autores y editoriales, y que permita su descarga en formato PDF de manera segura, resolverá la problemática actual de la falta de acceso eficiente y organizado a materiales educativos y culturales. Esta solución tecnológica fomentará la conservación del patrimonio bibliográfico local y asegurará su disponibilidad a largo plazo, gracias a la implementación de medidas de seguridad que protegerán los archivos contra accesos no autorizados o modificaciones maliciosas.

Asimismo, se espera que la biblioteca digital facilite el aprendizaje autónomo y la capacitación de los empleados, promoviendo un uso responsable de los recursos educativos digitales. La plataforma permitirá a los usuarios localizar materiales de manera rápida y precisa, lo cual contribuirá a un uso más eficiente del tiempo y los recursos disponibles.

Por lo tanto, al implementar esta solución, se logrará un impacto positivo tanto en la experiencia de los usuarios como en la gestión interna de los recursos, demostrando que la digitalización de contenidos no solo es una tendencia, sino una necesidad para garantizar el acceso al conocimiento en un contexto cada vez más dependiente de las herramientas tecnológicas.

3.1 Variable independiente:

Desarrollo e implementación de la biblioteca digital centralizada.

3.2 Variable dependiente:

Optimización en la gestión y distribución de recursos bibliográficos, mejora en la accesibilidad y preservación del conocimiento local.

IV.DESARROLLO DEL PROYECTO

4.1 Marco teórico

El marco teórico, en el desarrollo de este proyecto, ha sido fundamental para sustentar y justificar las decisiones tomadas a lo largo de la creación de la plataforma digital. Desde mi perspectiva, un marco teórico es el responsable de analizar críticamente los conceptos previos que dan forma a su desarrollo en el contexto de un proyecto destinado a la creación de una plataforma de gestión y consulta de recursos bibliográficos. Adicionalmente, es responsable de identificar áreas de conocimiento incompletas y territorios que aún no han sido explorados, lo que respalda la justificación de la relevancia del proyecto. Además, ayuda en la formulación de preguntas de investigación y la interpretación de los hallazgos.

Desde un enfoque académico, el marco teórico es definido como “El conjunto de conceptos, teorías y antecedentes que sustentan un proyecto de investigación o desarrollo. Su función principal es guiar la investigación al proporcionar una base sólida sobre la cual se estructura el trabajo, explicando los términos clave, los enfoques metodológicos y las referencias científicas que enmarcan el problema y sus soluciones”. (Hernández Sampieri, 2014)

4.1.1 Desarrollo de software

Es el proceso mediante el cual se crean aplicaciones o sistemas informáticos que resuelven necesidades específicas. Implica varias etapas, como análisis, diseño, programación, pruebas e implementación, con el fin de construir soluciones digitales funcionales, escalables y eficientes.

El desarrollo de software es un proceso sistemático que implica la creación, diseño, implementación y mantenimiento de aplicaciones informáticas. Para este proyecto, se utiliza un enfoque basado en arquitectura de tres capas (front-end, back-end y base de datos), que permite separar las responsabilidades y facilitar la escalabilidad del sistema (Sommerville, 2016).

4.1.2 Arquitectura de tres capas

Es una forma de estructurar una aplicación dividiéndola en tres niveles principales: presentación, lógica de negocio y acceso a datos. Esta separación permite que cada capa tenga responsabilidades claras, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

El modelo de arquitectura de tres niveles, que es el marco fundamental para el modelo de diseño lógico, segmenta los componentes de una aplicación en tres niveles de servicios. Estos niveles no corresponden necesariamente a ubicaciones físicas en varios equipos de una red, sino a capas lógicas de la aplicación. Cómo se distribuyen las partes de una aplicación en una topología física, según los requisitos del sistema (Microsoft, 2023).

4.1.2.1 Front-end

Es la parte visual de una aplicación, es decir, todo con lo que el usuario interactúa directamente. Incluye interfaces gráficas, botones, formularios y estilos. El objetivo del front-end es ofrecer una experiencia de usuario clara, intuitiva y funcional.

Es la capa de presentación que interactúa directamente con el usuario. Para este proyecto, se utiliza Angular, un framework de desarrollo web basado en TypeScript que permite crear interfaces dinámicas y responsivas (Angular, Angular.dev, 2023).

4.1.2.2 Angular

En el desarrollo de software existen diferentes aplicaciones y entornos de desarrollo para agilizar la creación de proyectos de software. Angular es uno de los Frameworks más famosos y utilizados en el entorno de desarrollo de software.

Framework de desarrollo web de código abierto basado en TypeScript, mantenido por Google, que facilita la creación de aplicaciones web SPA (Single Page Application) mediante un enfoque declarativo y basado en componentes (Angular, Angular.dev, 2023).

Figura 17 Logo de Angular.



Referencia. Obtenido de <https://www.npmjs.com/>

4.1.2.3 Back-end

Es el de procesar los datos, aplicar reglas de negocio, comunicarse con la base de datos y gestionar la lógica del sistema. El usuario no ve el back-end, pero es esencial para el funcionamiento de toda la aplicación. Es una pieza fundamental ya que en él se crea toda la lógica del funcionamiento principal del proyecto.

Es la capa de lógica de negocio que procesa las solicitudes del usuario y gestiona los datos. Se utiliza NestJS, un framework de Node.js que facilita la creación de APIs RESTful escalables y modulares (NestJS D. , 2023).

4.1.2.4 NestJS

Dentro del desarrollo BackEnd existen herramientas que proporcionar una estructura predeterminada para empezar a desarrollar la lógica del proyecto. NESTJS es la herramienta especializada en BackEnd que permite desarrollar back-ends escalables, mantenibles y bien organizados, ideales para crear APIs robustas.

Framework para construir aplicaciones del lado del servidor eficientes y escalables en Node.js, que utiliza TypeScript y está inspirado en Angular, facilitando la arquitectura modular (NestJS, 2025).

Figura 24 Logo de NestJS.



Referencia. Obtenido de <https://nestjs.com/>

4.1.2.5 TypeScript

Existen diferentes tipos de lenguaje de programación especializados para el desarrollo BackEnd y TypeScript es uno de ellos. Ayuda a detectar errores en tiempo de desarrollo, mejora la legibilidad del código y permite escribir aplicaciones más estructuradas y seguras, especialmente en proyectos grandes.

Superconjunto de JavaScript desarrollado por Microsoft que añade tipado estático opcional y otras características avanzadas al lenguaje, mejorando el desarrollo y mantenimiento de código (TypeScript, 2025).

Figura 30 Logo de Typescrip.



Referencia. Obtenido de <https://thenewstack.io/>

4.1.2.6 JavaScript

Al igual que Typescript, es fundamental para desarrollar páginas web interactivas y dinámicas, y hoy en día también se usa del lado del servidor gracias a tecnologías como Node.js.

Es un lenguaje ligero, interpretado y orientado a objetos con funciones de primera clase, y mejor conocido como el lenguaje de programación para las páginas Web, pero también se utiliza en muchos entornos que no son de navegador. Es un lenguaje de scripts que es dinámico, multiparadigma, basado en prototipos y admite estilos de programación orientados a objetos, imperativos y funcionales (MDN, 2025).

Figura 34 Logo de Java Script.



Referencia. Obtenido de <https://www.sanity.io/>

4.1.2.7 NodeJS

Existen lo llamados entornos de ejecución que permiten crear aplicaciones escalables y rápidas, especialmente en proyectos que manejan múltiples conexiones simultáneas, como APIs o servicios en tiempo real.

Entorno de ejecución para JavaScript construido con el motor V8 de Chrome que permite ejecutar código JavaScript en el servidor, creando aplicaciones web escalables (NodeJS, 2025).

Figura 38 Logo de NodeJS.



Referencia. Obtenido de <https://rubensa.wordpress.com/>

4.1.2.8 NPM

Es el sistema de gestión de paquetes de Node.js. Permite instalar, actualizar y gestionar dependencias de software necesarias para un proyecto.

Node Package Manager, el gestor de paquetes por defecto para Node.js, que permite a los desarrolladores instalar, compartir y gestionar dependencias de código (npmjs, 2025).

Figura 46 Logo de NPM.



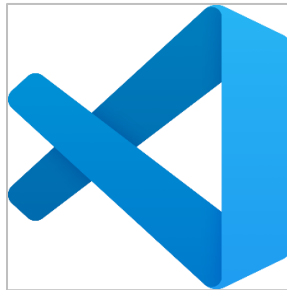
Referencia 1. Obtenido de <https://blog.packagecloud.io/>

4.1.2.9 Visual Studio Code

Para facilitar un entorno de desarrollo más eficiente y amigable para los desarrolladores, existen los editores de código que proporcionan mayor facilidad a la hora de estructuras y escribir código. Visual Studio Code es el mayor referente en editores de código fuente modernos y con la característica principal que es altamente personalizable.

Editor de código fuente desarrollado por Microsoft, gratuito y de código abierto, que ofrece potentes funcionalidades para desarrollo web y soporta múltiples lenguajes (Studio, 2025).

Figura 50 Logo de Visual Studio Code



Referencia. Obtenido de <https://www.aluracursos.com/>

4.1.2.10 Control de versiones

Para llevar un control en los avances realizados por un equipo desarrollador, tener presente un sistema que permite registrar todos los cambios realizados en el código de un proyecto a lo largo del tiempo, es fundamental para un proyecto de desarrollo de software.

Sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas posteriormente (Atlassian, What is version control?, 2024).

4.1.2.11 GIT

Es una herramienta de control de versiones distribuida que permite gestionar y registrar cambios en el código fuente.

Sistema de control de versiones distribuido diseñado para manejar desde pequeños a muy grandes proyectos con velocidad y eficiencia, enfocado en la integridad de los datos (Git, 2025).

Figura 58 Logo de Git.



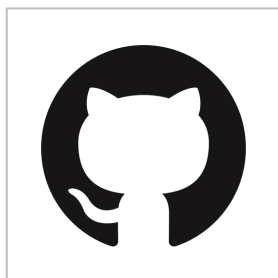
Referencia. Obtenido de <https://git-scm.com/>

4.1.2.12 GitHub

Para llevar a cabo un control de versiones, es necesario contar con una herramienta que e proporcione de manera visual el avance de cada uno de los puntos clave del proyecto, es posero que GitHub es el referente en plataformas en línea que utiliza Git para almacenar y gestionar proyectos de software.

Plataforma de desarrollo colaborativo basada en Git que permite alojar y revisar código, gestionar proyectos y construir software junto con otros desarrolladores (GitHub, 2024).

Figura 66 Logo de GitHub.



Referencia. Obtenido de <https://seeklogo.com/>

4.1.2.13 API RESTful:

A la hora de conectar proyecto de Frontend y Backend es necesario tener un intermediario permita la comunicación entre aplicaciones a través del protocolo HTTP, utilizando principios REST.

Una API RESTful es una interfaz de programación de aplicaciones que sigue los principios de REST (Transferencia de Estado Representacional).

Utiliza métodos HTTP (GET, POST, PUT, DELETE) para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre recursos identificados por URLs (Fielding, 2000).

4.1.3 Bases de datos

Para todo programa, proyecto, servicio o trabajo en donde se manejen grandes cantidades de datos recurrente, es necesario contar con un espacio de alojamiento el cual pueda ser accesible y permita realizar modificaciones y peticiones para alterar o modificar su contenido.

Es la capa de almacenamiento donde se gestionan los datos. Se utiliza MariaDB, un sistema de gestión de bases de datos relacionales que garantiza la integridad y eficiencia de los datos (Foundation, 2009 - 2025).

4.1.3.1 Bases de datos relacionales

Son sistemas que permiten almacenar, organizar y acceder a grandes volúmenes de información de manera eficiente. En el desarrollo de software, las bases de datos son fundamentales para gestionar los datos de los usuarios, productos, procesos, entre otros elementos clave de una aplicación.

La base de datos relacional mantendrá los datos de forma ordenada y accesible al contrario de tener alojado datos de este tipo dentro de bases de datos no relaciones o documentales, ya que este mantendrá el orden de los datos en tablas o interfaces mucho más accesibles, sin mencionar la seguridad y escalabilidad que este tipo de base de datos provee a la aplicación.

4.1.3.2 Beneficio de una base de datos relacional

Permiten organizar los datos en tablas con relaciones lógicas entre ellas, lo que asegura consistencia, integridad y evita duplicación innecesaria.

Una base de datos relacional tiene la importante ventaja de ser fácil de extender. Después de la creación original de una base de datos, una nueva categoría de datos se puede añadir sin necesidad de que todas las aplicaciones existentes sean modificadas. (Rouse, 2021).

4.1.3.3 Modelo de entidad relación (ER)

Es una representación gráfica de los elementos clave de una base de datos: entidades (tablas), atributos (columnas) y las relaciones entre ellas.

El modelo ER es una herramienta de diseño que representa las entidades, sus atributos y las relaciones entre ellas. Es fundamental para el diseño de bases de datos relacionales (Chen, 1976).

4.1.3.4 SQL (Structured Query Language)

Al igual que los lenguajes de programación, existen otro tipo de lenguajes especializados en interactuar con bases de datos relacionales y poder realizar consultas para alterar o modificar el contenido deseado. SQL es el mayor referente en cuanto a lenguaje de consulta en bases de datos relacionales se refiere.

El lenguaje de consulta estructurada (SQL) es un lenguaje de programación para almacenar y procesar información en una base de datos relacional. Una base de datos relacional almacena información en forma de tabla, con filas y columnas que representan diferentes atributos de datos y las diversas relaciones entre los valores de datos. Puede usar las instrucciones SQL para almacenar, actualizar, eliminar, buscar y recuperar información de la base de datos. También puede usar SQL para mantener y optimizar el rendimiento de la base de datos (Services), 2024).

4.1.3.5 MariaDB

Es un sistema de gestión de bases de datos relacional derivado de MySQL, conocido por su rendimiento, seguridad y compatibilidad.

Sistema de gestión de bases de datos relacional, fork de MySQL, desarrollado por la comunidad como software libre y de código abierto, compatible con MySQL (MariaDB, 2025).

Figura 70 Logo de MariaDB.



Referencia. Obtenido de <https://mariadb.org/>

4.1.3.6 HeidiSQL

Para un mejor manejo de los datos en un proyecto de software, se implementan herramientas graficas que permiten gestionar bases de datos como MariaDB, MySQL y PostgreSQL de manera intuitiva.

Herramienta gratuita de administración de bases de datos para MySQL, MariaDB, Microsoft SQL Server y PostgreSQL, que permite visualizar y editar datos fácilmente (HeidiSQL, 2025).

Figura 78 Logo de HeidiSQL.



Referencia. Obtenido de <https://www.heidisql.com/>

4.1.3.7 Postman

A la hora de realizar consultas o servicios de prueba de un proyecto de software, es ideal contar con una herramienta para probar y documentar APIs.

Plataforma API que permite a los desarrolladores diseñar, probar, documentar y compartir APIs, facilitando el proceso de desarrollo y colaboración entre equipos (Postman, 2025).

Figura 82 Logo de Postman.



Referencia. Obtenido de <https://howtocheckversion.com/>

4.1.4 Seguridad informática

Es el conjunto de prácticas y tecnologías que se implementan para proteger los sistemas, datos y redes frente a accesos no autorizados, ataques o pérdidas de información.

La seguridad informática es un aspecto crítico en el desarrollo de aplicaciones web, ya que protege los datos y sistemas contra accesos no autorizados y vulnerabilidades. Algunas de las prácticas de seguridad implementadas en este proyecto incluyen:

4.1.4.1 Autenticación y autorización

Para controlar quién puede ingresar al sistema y qué puede hacer cada usuario, se ha implementado un sistema de autenticación basado en tokens JWT. Este mecanismo permite validar la identidad de los usuarios registrados y asegurar que solo aquellos con los permisos adecuados accedan a recursos protegidos.

Se implementa un sistema de autenticación basado en tokens (JWT) para garantizar que solo usuarios registrados puedan acceder a los recursos (Auth0, What is JSON Web Token?, 2023).

4.1.4.2 Cifrado de datos

Con el objetivo de proteger la información que se transmite entre el cliente y el servidor, se utiliza el protocolo HTTPS. Esto permite cifrar los datos en tránsito, evitando que puedan ser interceptados o manipulados por terceros.

Se utiliza HTTPS para cifrar la comunicación entre el cliente y el servidor, protegiendo la información sensible (Encrypt, What is HTTPS?, 2023).

4.1.4.3 JWT (JSON Web Tokens)

Los JSON Web Tokens (JWT) son una herramienta clave para la autenticación y autorización en aplicaciones web modernas. Se trata de tokens compactos, estructurados en tres partes (encabezado, carga útil y firma), que permiten identificar a los usuarios de manera segura sin necesidad de almacenar sesiones en el servidor.

Los JWT son tokens compactos y autónomos que se utilizan para la autenticación y autorización en aplicaciones web. Están compuestos por tres partes: encabezado, carga útil y firma (Auth0, JSON Web Tokens, 2023).

4.1.5 Diseño centrado en el usuario (UX)

Es un enfoque de diseño que pone como prioridad las necesidades, expectativas y comportamientos del usuario. Busca crear interfaces intuitivas, accesibles y agradables que faciliten la interacción con la aplicación, mejorando así la experiencia general del usuario.

El diseño centrado en el usuario se enfoca en crear productos que satisfagan las necesidades y expectativas de los usuarios. Incluye principios como la usabilidad, accesibilidad y retroalimentación (Norman, 2014).

4.1.5.1 Angular Material

Su objetivo es facilitar la construcción de interfaces modernas y funcionales siguiendo los principios de Material Design de Google. Esta herramienta proporciona una gran variedad de elementos visuales prediseñados como botones, formularios, menús y tarjetas, todos con una apariencia coherente y profesional. Gracias a Angular Material, se agiliza el desarrollo del frontend sin sacrificar la estética ni la experiencia de usuario.

Biblioteca de componentes UI para Angular que implementa los principios de Material Design de Google, ofreciendo elementos visuales prediseñados y consistentes para crear interfaces modernas (Angular, Angular Material, 2025).

Figura 91 Angular Material Logo.



Referencia. Obtenido de <https://docs.angular.lat/>

4.1.5.2 Tailwind

Esta metodología ayuda a desarrollar más rápido, ya que cada clase está diseñada para cumplir una función específica (como márgenes, colores, tipografía, etc.), permitiendo crear diseños flexibles y reutilizables con facilidad.

Framework CSS utilitario que permite construir diseños personalizados sin escribir CSS, mediante clases predefinidas que se aplican directamente en el HTML, favoreciendo un desarrollo más rápido (Tailwind, 2025).

Figura 99 Logo Tailwind CSS



Referencia. Obtenido de <https://codersfree.com/>

4.1.5.3 CSS

Junto con HTML, CSS es fundamental para construir interfaces agradables y bien estructuradas, ya que se encarga de toda la parte visual del diseño.

Lenguaje de hojas de estilo en cascada usado para describir la presentación de documentos HTML, definiendo cómo deben mostrarse los elementos en pantalla, papel u otros medios (W3C, 2023).

Figura 103 Logo CSS.



Referencia. Obtenido de <https://www.adaweb.es/>

4.1.5.4 HTML

HTML (HyperText Markup Language) es el lenguaje de marcado que se encarga de estructurar el contenido de una página web. Define los distintos elementos que aparecen en pantalla, como textos, imágenes, botones o formularios, utilizando etiquetas.

Lenguaje de marcado estándar utilizado para crear páginas web y aplicaciones web, que define la estructura y contenido básico de una página mediante etiquetas (WHATWG, 2025).

Figura 107 Logo HTML



Referencia. Obtenido de <https://www.adaweb.es/>

4.1.5.5 Figma

En las fases de creación y desarrollo de un proyecto de software, se encuentra el maquetado y diseño interfaces, botones, inputs, etc. Siendo un paso crucial a la hora de estructurar y definir bien un proyecto. Figma es el referente más famoso en la creación de diseños web. Es una herramienta de diseño colaborativo en línea que permite crear interfaces de usuario (UI), prototipos interactivos y sistemas de diseño.

Herramienta de diseño de interfaces basada en la nube que permite crear, colaborar y prototipar diseños de UI/UX en tiempo real con múltiples usuarios simultáneamente (Figma.com, 2025).

Figura 115 Logo de Figma



Referencia. Obtenido de <https://github.com/figma>

4.1.5.6 Responsive desing:

La adaptabilidad a diferentes tamaños para u0 a página o aplicación web es crucial para que dicho proyecto sea funcionas y accesible a cualquier usuario que utilice cualquier tipo de dispositivo. El diseño responsivo es una técnica de diseño web que permite que una aplicación se adapte automáticamente a diferentes tamaños de pantalla, como móviles, tabletas y escritorios.

El diseño responsivo asegura que las interfaces se adapten a diferentes dispositivos y tamaños de pantalla, mejorando la experiencia del usuario (Marcotte, 2011).

4.1.5.7 Accesibilidad:

Se refiere a la capacidad de una aplicación para ser utilizada por personas con diferentes capacidades. La accesibilidad en el diseño garantiza que las interfaces sean utilizables por personas con discapacidades. Incluye prácticas como el uso de texto alternativo y contraste adecuado (W3C, 2023).

4.1.6 Metodología de trabajo en el desarrollo de software

“Es el conjunto de técnicas y métodos que se utilizan para diseñar una solución de software. Es importante señalar que existen varias, de manera que es decisión del equipo cual implementarla” (Admltsqmet, 2022).

4.1.7 Metodologías ágiles

Son enfoques de gestión de proyectos que priorizan la flexibilidad, la colaboración y la entrega continua de valor.

Por definición, las metodologías ágiles son aquellas que permiten adaptar la forma de trabajo a las condiciones del proyecto, consiguiendo flexibilidad e inmediatez en la respuesta para amoldar el proyecto y su desarrollo a las circunstancias específicas del entorno. En esencia, las empresas que apuestan por esta metodología consiguen gestionar sus proyectos de forma flexible, autónoma y eficaz reduciendo los costes e incrementando su productividad (Sotomayor, 2024).

4.1.7.1 Scrumban:

Es una metodología que combina lo mejor de Scrum y Kanban.

La metodología scrumban combina las mejores características de scrum y kanban en un marco de gestión de proyectos híbrido. Utiliza la estructura estable del scrum: sprints, reuniones rápidas y retrospectivas. Por otro lado, incorpora el flujo de trabajo visual y las limitaciones del trabajo en curso del kanban. El resultado es un método verdaderamente flexible para gestionar proyectos de cualquier tamaño (Atlassian, Scrumban: domina dos metodologías ágiles, s.f.).

4.1.8 Pruebas unitarias

Las pruebas unitarias ayudan a detectar errores temprano, facilitan el mantenimiento del sistema y aseguran que futuras modificaciones no afecten el comportamiento esperado del software. El primer nivel de las pruebas es el de la prueba unitaria o de componente que consiste en la verificación de unidades de software de forma aislada, es decir, probar el correcto funcionamiento de una unidad de código (Peño, 2015).

V. METODOLOGÍA

5.1 Requerimientos Funcionales

5.1.1 Gestión de Usuarios

- Los administradores deben poder gestionar el acceso de los usuarios (habilitar/deshabilitar usuarios).
- Los usuarios solo podrán acceder a los libros si están registrados.

5.1.2 Visualización de Libros

- Los usuarios deben poder navegar por la lista de libros disponibles en la biblioteca.
- Los libros deben estar categorizados por **categoría, autor y editorial**.
- Se debe incluir un buscador para filtrar libros por título, categoría o autor.

5.1.3 Descarga de Libros

- Los usuarios deben tener la opción de descargar los libros en formato PDF.
- Se debe registrar en la base de datos cuándo un libro ha sido descargado.

5.1.4 Gestión de Libros (Administrador)

- Los administradores deben poder agregar, editar y eliminar libros de la base de datos.
- Se debe permitir cargar archivos PDF de los libros al sistema.
- Los administradores deben poder asignar categorías y autores a cada libro.

5.1.5 Visualización de Detalles del Libro

- Cada libro debe mostrar información detallada, como título, autor, editorial, categoría y código.
- Mostrar un botón de descarga destacado en la página de detalles del libro.

5.1.6 Categorías y Autores

- Los administradores deben poder gestionar categorías y autores (crear, editar y eliminar).

5.1.7 Panel de Administración

- Los administradores deben tener acceso a un panel donde puedan gestionar usuarios, libros, categorías y autores.
- Visualizar estadísticas básicas, como el número de descargas por libro.

5.1.8 Seguridad

- Implementar validaciones para evitar la carga de archivos maliciosos.
- Limitar las descargas simultáneas por usuario para evitar abusos.

5.1.9 Usabilidad

- La interfaz debe ser intuitiva, con botones y enlaces claros para realizar acciones como buscar, descargar y visualizar libros.
- Utilizar colores neutros (color insignia de la Fiscalía) y destacar las opciones principales con colores llamativos para facilitar la navegación.

5.1.10 Rendimiento

- Las búsquedas y cargas de libros deben realizarse en menos de 2 segundos para garantizar una buena experiencia de usuario.
- Optimizar las descargas para que no excedan el ancho de banda disponible en el servidor local.

5.1.11 Responsivo

- La aplicación debe ser responsiva, adaptándose a diferentes dispositivos como computadoras, tablets y smartphones.
- Soporte para los navegadores web modernos (Chrome, Firefox, Edge, etc.).

5.1.12 Seguridad

- Asegurarse de que los archivos PDF no contengan contenido malicioso antes de permitir su carga.

5.1.13 Escalabilidad

- Diseñar la base de datos y el sistema para que puedan manejar un crecimiento en la cantidad de libros y usuarios sin comprometer el rendimiento.

5.1.14 Mantenibilidad

- La aplicación debe ser modular, permitiendo futuras actualizaciones o integración de nuevas funcionalidades.

5.1.15 Disponibilidad

- Asegurar que el sistema esté disponible en horarios laborales definidos y proporcione mensajes claros en caso de que esté en mantenimiento.

5.1.16 Auditoría

Registrar todas las acciones críticas, como inicio de sesión, descargas y modificaciones en los datos, para futuras auditorías.

5.1.17 Cronograma de actividades

Actividades para realizar para el desarrollo del proyecto, basado en un cronograma de actividades planteado a inicio del proyecto y con la verificación y aprobación del asesor empresarial y el asesor académico.

Tabla 1 Cronograma de Actividades

Actividad programada	Semanas														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Definición del proyecto															
Datos generales de la empresa.															
Antecedentes de la empresa															
Planteamiento del problema.															
Justificación.															
Planteamiento de objetivos.															
Hipótesis.															
Marco teórico.															

Metodología.															
Desarrollo del proyecto.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Levantamiento de requerimientos funcionales y no funcionales.															
Creación de diseño de interfaz de usuario y definición de la paleta de colores															
Configuración del entorno local de desarrollo (Nest.JS, Angular, MariaDB)															
Diseño de la base de datos.															
Configuración inicial de las entidades en Nest.JS.															
Creación de módulos y servicios básicos en Nest.JS (CRUD para libros, autores, categorías).															
Desarrollo del proyecto.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Diseño inicial del Front-end en Angular (interfaz de usuario para gestión de libros).															
Integración del Back-end y Front-end.															
Desarrollo de la funcionalidad para subir y descargar PDFs.															

Configuración de módulos adicionales para los usuarios.															
Pruebas unitarias y funcionales iniciales.															
Corrección de errores reportados.															
Revisión del proyecto completo.															
Implementación en el servidor local.															
Presentación oficial del proyecto y cierre.															
Conclusión del proyecto	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Análisis de resultados.															
Redacción de conclusiones.															
Oficio de dictamen															
Impresión y entrega de memoria															

Para el desarrollo del presente proyecto se optó por la metodología Scrumban, una combinación entre los marcos ágiles Scrum y Kanban. Esta metodología fue seleccionada debido a su flexibilidad, capacidad de adaptación y control visual del flujo de trabajo, lo que resulta altamente efectivo en proyectos que requieren entregas iterativas y mejoras constantes, como en el caso del diseño y desarrollo de una plataforma web para gestión bibliográfica digital.

Scrumban permite gestionar tareas en curso, priorizar funcionalidades críticas y adaptarse a nuevas necesidades surgidas durante la implementación. La iteración continua y la priorización dinámica se ajustaron perfectamente al entorno del proyecto, dado que:

- Se requería un sistema que permitiera realizar ajustes sobre la marcha conforme se descubrían nuevas necesidades institucionales.
- Existía la posibilidad de recibir retroalimentación constante por parte de los usuarios (asesores institucionales y académicos).
- Se necesitaba una herramienta visual para monitorear el avance, distribuir responsabilidades y controlar la calidad del software.

El ciclo de trabajo se organizó mediante un tablero de actividades dividido por semanas, donde cada semana representaba una columna de avance (Backlog, En Progreso, En Revisión, Finalizado). Las tareas se distribuyeron en épicas, historias de usuario, tareas técnicas y subtareas, siguiendo la estructura vista en herramientas como Jira y Notion.

5.1.18 Herramientas y materiales necesarios

- **Lenguajes de programación:** TypeScript (para NestJS y Angular), SQL (para MariaDB).
- **Frameworks y libraries:** NestJS (back-end), Angular (front-end), Bootstrap (diseño responsive).
- **Base de datos:** MariaDB, con herramientas de gestión como phpMyAdmin o DBeaver.
- **Control de versiones:** Git, con un repositorio alojado en GitHub o GitLab.
- **Gestión de proyectos:** Trello o Jira para la organización de tareas y seguimiento del progreso.
- **Servidor local:** Configuración de un servidor con Debian para el despliegue inicial de la plataforma.

5.1.19 Seguimiento y control

A lo largo de cada semana (sprint), se realizarán reuniones diarias (Daily Stand-ups) para revisar el progreso de las tareas, identificar obstáculos y ajustar el plan de trabajo según sea necesario. El tablero Kanban se actualizará continuamente para reflejar el estado actual de las tareas, lo que permitirá al equipo mantener un flujo de trabajo constante y transparente.

5.1.20 Revisión y mejora continua

Al final de cada sprint, se llevará a cabo una reunión de revisión (Sprint Review) para presentar las funcionalidades implementadas y recibir retroalimentación.

Posteriormente, se realizará una reunión de retrospectiva (Sprint Retrospective) para identificar áreas de mejora y ajustar el proceso de trabajo en los siguientes sprints.

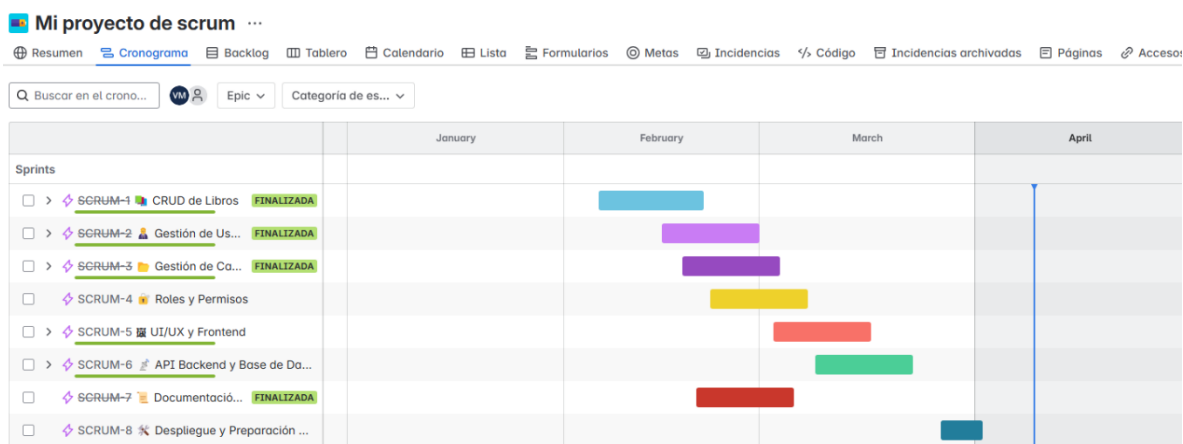
La metodología Scrumban, combinada con un plan de trabajo detallado y organizado por semanas, permite una ejecución eficiente y controlada del proyecto.

A través de la implementación iterativa de funcionalidades y la revisión constante del progreso, se garantiza que la plataforma cumpla con los objetivos establecidos y brinde un servicio de calidad a los empleados de la Fiscalía. Este enfoque asegura la entrega de un producto robusto, seguro y escalable dentro del plazo definido.

Esto nos permitió:

- Una gestión eficiente del tiempo y recursos.
- Evitar la sobrecarga de trabajo.
- Revisión ágil y continua de las funcionalidades implementadas.
- Mejora constante basada en retroalimentación.

Figura 119 Tablero Scrum en la aplicación Jira



Referencia. Elaboración propia en jira.com

El proyecto fue calendarizado para un periodo de 12 semanas, desde el 13 de enero hasta el 4 de abril. Cada semana funcionó como un sprint, estableciendo entregables parciales con revisiones al finalizar. Se establecieron tareas específicas en un tablero Kanban, dividido en columnas: Por hacer, En progreso y Terminado, permitiendo una clara visualización del flujo de trabajo.

5.1.21 Herramientas y materiales utilizados

- **Lenguajes:** TypeScript, SQL.
- **Frameworks:** NestJS (Back), Angular (Front), TailwindCSS.
- **Base de datos:** MariaDB.
- **Control de versiones:** Git + GitHub.
- **Plataforma de gestión:** Jira (Scrumban) para organización de tareas y progreso.
- **Servidor de pruebas:** Debian local (localhost).
- **Diseño UI:** Figma como guía de estilos oficial del sistema.

5.1.22 Seguimiento y control

Cada sprint se complementó con una revisión de tareas (Sprint Review) y retroalimentación continua con el asesor empresarial y académico. Se llevaron a cabo pruebas funcionales para validar el cumplimiento de los objetivos, incluyendo pruebas unitarias en componentes del Front-End, así como simulaciones en ambientes controlados para validar el flujo completo del usuario.

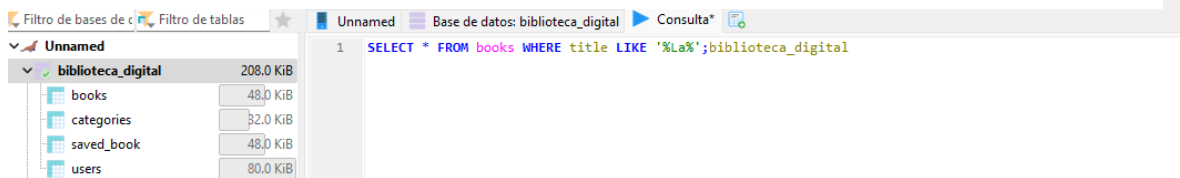
5.2 Base de Datos

El sistema utilizará MariaDB, una bifurcación de MySQL, optimizada para integrarse con Spring Boot. La base de datos se diseñó bajo un enfoque relacional y se normalizó hasta 3FN (Tercera Forma Normal) para evitar redundancias e inconsistencias.

5.2.1 Esquema General

- Books

Figura 127 Tablas de la Base de Datos.



Referencia. Creación propia en la herramienta grafica HeidiSQL.

La tabla books tiene la función de almacenar la inflación más relevante que contiene un libro, siendo editorial, file_path (ruta de almacenamiento del libro), title, cateogriald(llave foránea), autor, code, lastViewed, createdAt. Esto ayuda a agilizar el tiempo de petición de consultas de tipo POST, GET, PUT & DELETE.

Figura 131 Código CREATE para la tabla 'books'

```
1 CREATE TABLE `books` (  
2   `id` INT(11) NOT NULL AUTO_INCREMENT,  
3   `editorial` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',  
4   `file_path` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',  
5   `title` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',  
6   `categoryId` INT(11) NULL DEFAULT NULL,  
7   `author` VARCHAR(255) NULL DEFAULT NULL COLLATE 'utf8mb4_uca1400_ai_ci',  
8   `code` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',  
9   `lastViewed` TIMESTAMP NULL DEFAULT NULL,  
10  `createdAt` TIMESTAMP(6) NOT NULL DEFAULT current_timestamp(6),  
11  PRIMARY KEY (`id`) USING BTREE,  
12  UNIQUE INDEX `IDX_c19328bbdf15e7ddbea3812318` (`code`) USING BTREE,  
13  INDEX `FK_a0f13454de3df36e337e01dbd55` (`categoryId`) USING BTREE,  
14  CONSTRAINT `FK_a0f13454de3df36e337e01dbd55` FOREIGN KEY (`categoryId`) REFERENCES `categories` (`id`) ON UPDATE NO ACTION ON DELETE NO ACTION  
15 )  
16 COLLATE='utf8mb4_uca1400_ai_ci'  
17 ENGINE=InnoDB  
18 AUTO_INCREMENT=55  
19 ;  
20
```

Referencia. Generado en la consulta de la herramienta grafica "HeidiSQL".

Tabla 6 Tabla Books

Campo	Tipo de Dato	Restricciones
id	INT(11)	PRIMARY KEY, AUTO_INCREMENT
editorial	VARCHAR(255)	NOT NULL
file_path	VARCHAR(255)	NOT NULL
title	VARCHAR(255)	NOT NULL
categoryId	INT(11)	FOREIGN KEY (categories.id), NULLABLE
author	VARCHAR(255)	NULLABLE
code	VARCHAR(255)	NOT NULL, ÚNICO
lastViewed	TIMESTAMP	NULLABLE
createdAt	TIMESTAMP(6)	NOT NULL, DEFAULT current_timestamp(6)

Referencia. Elaboración propia en Microsoft Excel

- **categories**

Figura 139 Código CREATE par la tabla 'categories'.

```

1 CREATE TABLE `categories` (
2   `id` INT(11) NOT NULL AUTO_INCREMENT,
3   `nombre` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
4   PRIMARY KEY (`id`) USING BTREE,
5   UNIQUE INDEX `IDX_8053c619b9d0eaebeeeae2a0be` (`nombre`) USING BTREE
6 )
7 COLLATE='utf8mb4_uca1400_ai_ci'
8 ENGINE=InnoDB
9 AUTO_INCREMENT=14
10 ;

```

Referencia. Generado en la consulta de la herramienta grafica "HeidiSQL"

La tabla categorías, esta signada para almacenar el nombre de las diferentes categorías en existencia en la que pueda estar catalogado cada libro, contando únicamente con una sola columna llamada nombre. Esta almacena el nombre de las categorías, asignándoles un Id único, el cual se utilizará para asignarle un a categoría a cada libro individualmente o en conjunto.

Tabla 10 Categories

Campo	Tipo de Dato	Restricciones
id	INT(11)	PRIMARY KEY, AUTO_INCREMENT
nombre	VARCHAR(255)	NOT NULL, ÚNICO

Referencia. Elaboración propia en Microsoft Excel

- **saved_book**

Figura 143 Código CREATE para la tabla 'saved_book'.

```

1 CREATE TABLE `saved_book` (
2   `id` INT(11) NOT NULL AUTO_INCREMENT,
3   `createdAt` TIMESTAMP NOT NULL DEFAULT current_timestamp(),
4   `userId` INT(11) NULL DEFAULT NULL,
5   `bookId` INT(11) NULL DEFAULT NULL,
6   PRIMARY KEY (`id`) USING BTREE,
7   INDEX `FK_8681f8faa9a10d4a6e38b829c77` (`userId`) USING BTREE,
8   INDEX `FK_78029ada0e37dc746bbae2f2374` (`bookId`) USING BTREE,
9   CONSTRAINT `FK_78029ada0e37dc746bbae2f2374` FOREIGN KEY (`bookId`) REFERENCES `books` (`id`) ON UPDATE NO ACTION ON DELETE NO ACTION,
10  CONSTRAINT `FK_8681f8faa9a10d4a6e38b829c77` FOREIGN KEY (`userId`) REFERENCES `users` (`id`) ON UPDATE NO ACTION ON DELETE NO ACTION
11 )
12 COLLATE='utf8mb4_uca1400_ai_ci'
13 ENGINE=InnoDB
14 :

```

Referencia. Generado en la consulta de la herramienta grafica "HeidiSQL".

La tabla saved_book almacena los libros guardados por el usuario, funciona relacionando el id del usuario con el id de un libro en particular, para indicar que dicho libro ha sido guardad o por el usuario.

Tabla 18 Saved_book

Campo	Tipo de Dato	Restricciones
id	INT(11)	PRIMARY KEY, AUTO_INCREMENT
createdAt	TIMESTAMP	NOT NULL, DEFAULT current_timestamp()
userId	INT(11)	FOREIGN KEY (users.id), NULLABLE
bookId	INT(11)	FOREIGN KEY (books.id), NULLABLE

Referencia. Elaboración propia en Microsoft Excel

- users

Figura 147 Código CREATE para la tabla 'users'.

```

1 CREATE TABLE `users` (
2   `id` INT(11) NOT NULL AUTO_INCREMENT,
3   `email` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
4   `puesto` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
5   `area_adscpcion` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
6   `password` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
7   `username` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
8   `role` ENUM('admin','user','partner') NOT NULL DEFAULT 'user' COLLATE 'utf8mb4_uca1400_ai_ci',
9   `firstname` VARCHAR(255) NULL DEFAULT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
10  `lastname` VARCHAR(255) NULL DEFAULT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
11  `publicId` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
12  `sub_area` VARCHAR(255) NOT NULL COLLATE 'utf8mb4_uca1400_ai_ci',
13  PRIMARY KEY (`id`) USING BTREE,
14  UNIQUE INDEX `IDX_97672ac88f789774dd47f7c8be` (`email`) USING BTREE,
15  UNIQUE INDEX `IDX_fe0bb3f6520ee0469504521e71` (`username`) USING BTREE,
16  UNIQUE INDEX `IDX_9099c98f00a1b5aca6b8f7f04a` (`publicId`) USING BTREE
17 )
18 COLLATE='utf8mb4_uca1400_ai_ci'
19 ENGINE=InnoDB
20 AUTO_INCREMENT=23
21 ;

```

Referencia. Generado en la consulta de la herramienta grafica "HeidiSQL".

La tabla users ocupa un rol muy importante en la creación y desarrollo del proyecto. En ella se almacena la información de todos lo usuarios registrados en la Biblioteca digital. Entre los datos almacenado se encuentra el de 'role', el cual funciona al ahora de asignar roles de usuario y redireccionamiento de rutas a la hora de ingresar con las credenciales solicitadas.

Tabla 26 Users

Campo	Tipo de Dato	Restricciones
id	INT(11)	PRIMARY KEY, AUTO INCREMENT
email	VARCHAR(255)	NOT NULL, ÚNICO
puesto	VARCHAR(255)	NOT NULL
area_adscpcion	VARCHAR(255)	NOT NULL
password	VARCHAR(255)	NOT NULL
username	VARCHAR(255)	NOT NULL, ÚNICO
role	ENUM('admin','user','partner')	NOT NULL, DEFAULT 'user'
firstname	VARCHAR(255)	NULLABLE
lastname	VARCHAR(255)	NULLABLE
publicId	VARCHAR(255)	NOT NULL, ÚNICO
sub_area	VARCHAR(255)	NOT NULL

Referencia. Elaboración propia en Microsoft Excel

5.3 Estilos e Historia de Usuario

Como parte de la ingeniería de requerimientos, se identificaron y modelaron los diferentes actores que interactúan con el sistema, así como los casos de uso que cada uno puede ejecutar de acuerdo a su nivel de privilegios y responsabilidades dentro de la plataforma. A continuación, se describen de forma detallada los casos de uso asociados a cada tipo de usuario: Administrador (Admin), Socio (Partner) y Usuario Final (User).

5.3.1 Actor Administrador (Admin)

El Administrador representa el perfil con mayores privilegios dentro del sistema. Está encargado de la gestión integral del entorno administrativo, incluyendo el mantenimiento de las categorías del sistema, así como el alta, modificación y eliminación de usuarios. Entre las principales acciones que puede ejecutar se encuentran:

- **Iniciar sesión:** Acceso autenticado a través de credenciales válidas.
- **Ver dashboard de administración:** Visualización del panel de control que resume información relevante del sistema.
- **Gestionar categorías:**
 - Crear nuevas categorías.
 - Modificar categorías existentes.
 - Eliminar categorías obsoletas.
- **Gestionar usuarios:**
 - Crear nuevos usuarios asignando roles específicos.
 - Editar información de usuarios actuales.
 - Eliminar cuentas de usuarios que ya no deban tener acceso.
 - Visualizar la lista completa de usuarios registrados.
- **Cerrar sesión:** Finalizar la sesión activa del sistema.

Este actor es esencial para mantener la operatividad del sistema y realizar tareas de mantenimiento continuo sobre la estructura de datos y usuarios.

5.3.2 Actor: Socio Final (Partner)

El Partner es un tipo de usuario que posee un nivel intermedio de permisos. Este rol puede consultar información clave del sistema sin realizar alteraciones. Su interacción se limita principalmente a la revisión y monitoreo de la información.

- **Iniciar sesión:** Autenticarse para acceder a su espacio de trabajo.
- **Ver dashboard:** Consultar métricas o datos relevantes, sin acceso a modificaciones.
- **Consultar categorías:** Visualizar las categorías creadas en el sistema.
- **Consultar usuarios:** Acceso de solo lectura a la lista de usuarios registrados.
- **Cerrar sesión:** Finalizar su sesión en la plataforma.

Este rol es útil para colaboradores externos o aliados estratégicos que requieren consultar el estado de ciertos elementos sin comprometer la integridad del sistema.

5.3.3 Actor: Usuario final (User)

El Usuario Final es el perfil más común en la plataforma. Su experiencia está diseñada para consumir información y realizar acciones básicas relacionadas a su cuenta personal o contenido habilitado públicamente.

- **Iniciar sesión:** Acceder al sistema con sus credenciales personales.
- **Consultar perfil:** Revisar su propia información registrada.
- **Ver contenido público:** Navegar por las categorías visibles o secciones disponibles sin autenticación avanzada.
- **Cerrar sesión:** Finalizar sesión y cerrar su cuenta de forma segura.

Este actor es clave en la validación del sistema desde la perspectiva del consumidor final, permitiendo evaluar la usabilidad, eficiencia y experiencia de usuario.

5.3.4 Representación Gráfica del Modelo

@startuml

actor Admin
actor Partner
actor User

```
rectangle Sistema {  
    Admin --> (Iniciar sesión)  
    Admin --> (Ver dashboard de administración)  
    Admin --> (Gestionar categorías)  
    Admin --> (Gestionar usuarios)  
    Admin --> (Cerrar sesión)  
  
    (Gestionar categorías) --> (Crear categoría)  
    (Gestionar categorías) --> (Editar categoría)  
    (Gestionar categorías) --> (Eliminar categoría)  
  
    (Gestionar usuarios) --> (Crear usuario)  
    (Gestionar usuarios) --> (Editar usuario)  
    (Gestionar usuarios) --> (Eliminar usuario)  
    (Gestionar usuarios) --> (Ver lista de usuarios)  
  
    Partner --> (Iniciar sesión)  
    Partner --> (Ver dashboard de administración)  
    Partner --> (Consultar categorías)  
    Partner --> (Consultar usuarios)  
    Partner --> (Cerrar sesión)  
  
    User --> (Iniciar sesión)  
    User --> (Consultar perfil)  
    User --> (Ver contenido público)  
    User --> (Cerrar sesión)  
}
```

@enduml

5.4 Guía de Estilos

Con el fin de mantener la coherencia visual, la accesibilidad y la facilidad de mantenimiento de la interfaz gráfica del sistema desarrollado, se elaboró una Guía de Estilos como documento de referencia para los desarrolladores de frontend, diseñadores UX/UI y cualquier profesional involucrado en futuras mejoras del sistema.

La guía se diseñó sobre la plataforma **Figma**, herramienta que permitió definir los lineamientos gráficos del proyecto, estableciendo componentes reutilizables, variables de color, tipografía y estructuras visuales estandarizadas.

A continuación, se describen los principales elementos que componen la guía de estilo de la aplicación web:

5.4.1 Paleta de Colores

La selección de la paleta de colores se realizó en tomando como código de color **#213B7C**, siendo este el color insignia de la FGEO. En base él y a el concepto visual del sistema: profesional, accesible, moderno y enfocado a la gestión administrativa, la jerarquía cromática emplea tonos azules profundos para transmitir confianza, junto con acentos en colores neutros para la legibilidad.

5.4.1.1 Main_Color #213B7C:

Utilizado como color principal aplicado en todas las interfaces de la aplicación web, se tomó como referencia para crear la paleta de colores final y se implementa en diferentes componentes, como lo son sidebards, headers y títulos principales.

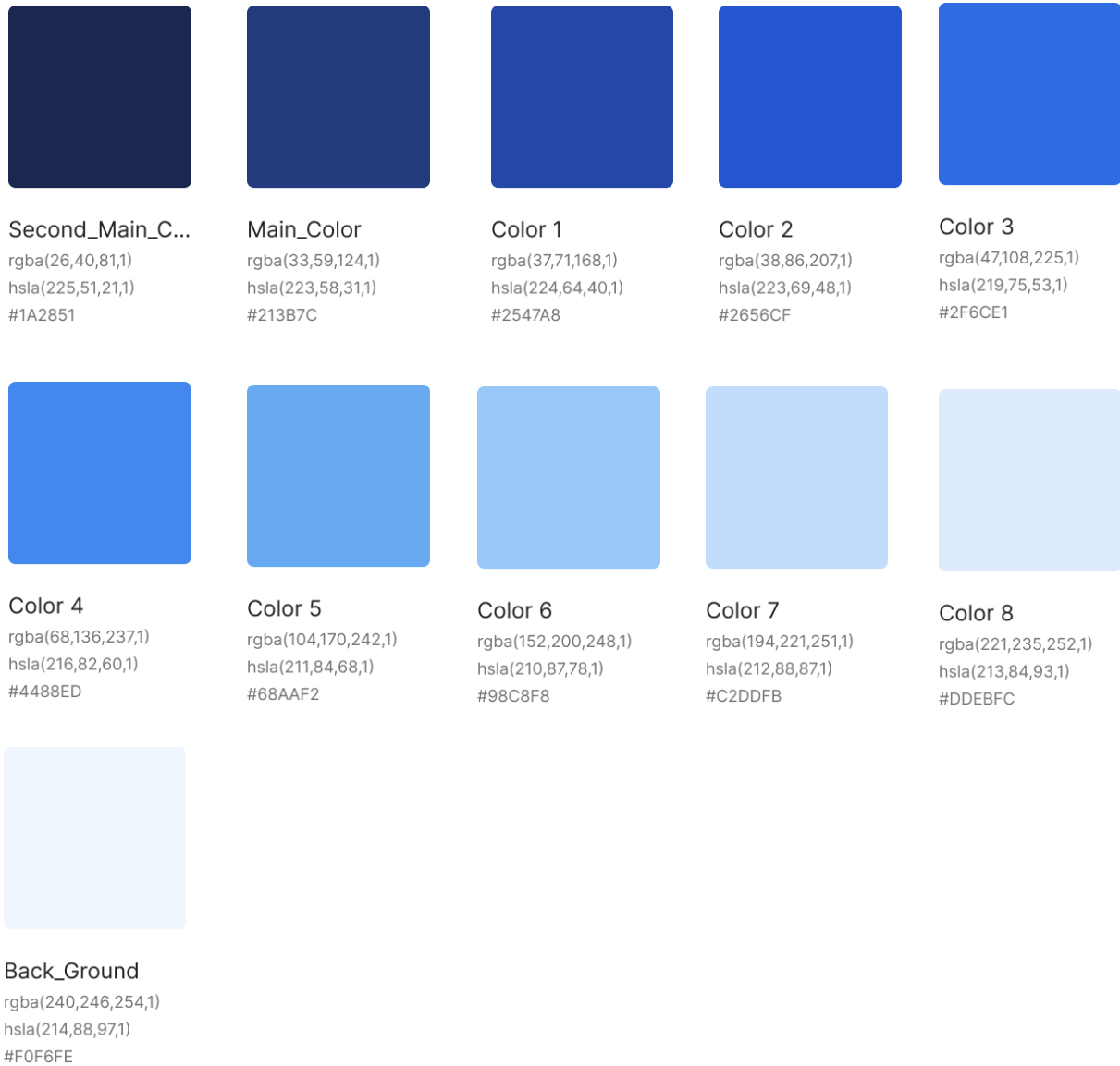
5.4.1.2 Second_Main_Color #1A2851:

Utilizado como un 2do color principal, el cual ayuda a resaltar más ciertos aspectos que sean importantes dentro de las interfaces, como lo son botones, textos de información e iconos.

5.4.1.3 Back_Ground #F0F6FE:

Utilizado como color de fondo para todas las interfaces de la página web, dándole un toque único y fresco a la vida del usuario.

Figura 151 Paleta de Colores.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.2 Tipografía

Se eligió la fuente Poppins por su claridad, legibilidad y estética moderna. Es una fuente sans-serif optimizada para la web que permite una lectura fluida en distintos tamaños.

Figura 155 Fuente Poppins.



Referencia. Obtenida de <https://fonts.google.com/>

FIGURA 162 OPTIONS REGULAR.

options		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Regular	None	
Size	Paragraph Spacing	
13px	0px	
Line Height	Case	
Automatic	Original	

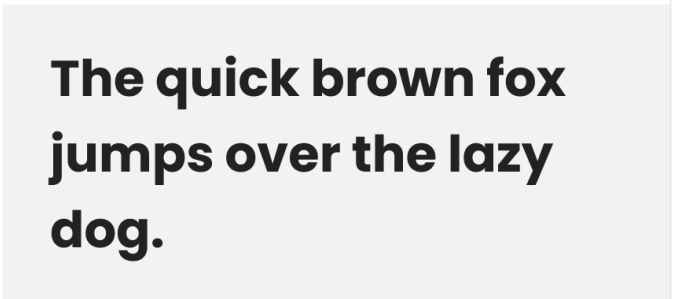
Referencia. Generados con la Herramienta de diseño "Figma"

FIGURA 172 TITLE2 MEDIUM.

title2		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Medium	None	
Size	Paragraph Spacing	
20px	0px	
Line Height	Case	
Automatic	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

FIGURA 168 TITLE.

Title		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Bold	None	
Size	Paragraph Spacing	
32px	0px	
Line Height	Case	
Automatic	Original	

Referencia. Generado con la Herramienta de diseño "Figma"

Figura 185 Info Light.

info		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Light	None	
Size	Paragraph Spacing	
13px	0px	
Line Height	Case	
Automatic	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

Figura 176 Text Regular.

text		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Regular	None	
Size	Paragraph Spacing	
12px	0px	
Line Height	Case	
Automatic	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

Figura 191 Title_book_preview ExtraBold.

Title_book_preview		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
ExtraBold	None	
Size	Paragraph Spacing	
12px	0px	
Line Height	Case	
Automatic	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

Figura 195 Info_book_preview.

info_book_preview		
Font	Letter Spacing	
Poppins	0%	
Weight	Text Decoration	
Regular	None	
Size	Paragraph Spacing	
10px	2px	
Line Height	Case	
Automatic	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

Figura 201. Buttons Medium

buttons		
Font	Letter Spacing	
Poppins	-8%	
Weight	Text Decoration	
Medium	None	
Size	Paragraph Spacing	
10px	2px	
Line Height	Case	
2px	Original	

Referencia. Generados con la Herramienta de diseño "Figma"

5.4.3 Inputs

Los campos de entrada de texto están diseñados para priorizar la accesibilidad visual y la respuesta del usuario. Se destacan por un diseño limpio con bordes redondeados y alto contraste cuando están enfocados.

Figura 207 Inputs de Login y Register.

Nombres	e-mail
Apellidos	Contraseña
E-mail	

Referencia. Generados con la Herramienta de diseño "Figma"

5.4.4 Botones

Los botones siguen un patrón coherente que permite diferenciarlos según su función (primaria, secundaria, peligro, desactivado).

Figura 211 Botones de Registro, Inicio de sesión, Categoría, eliminar y editar.



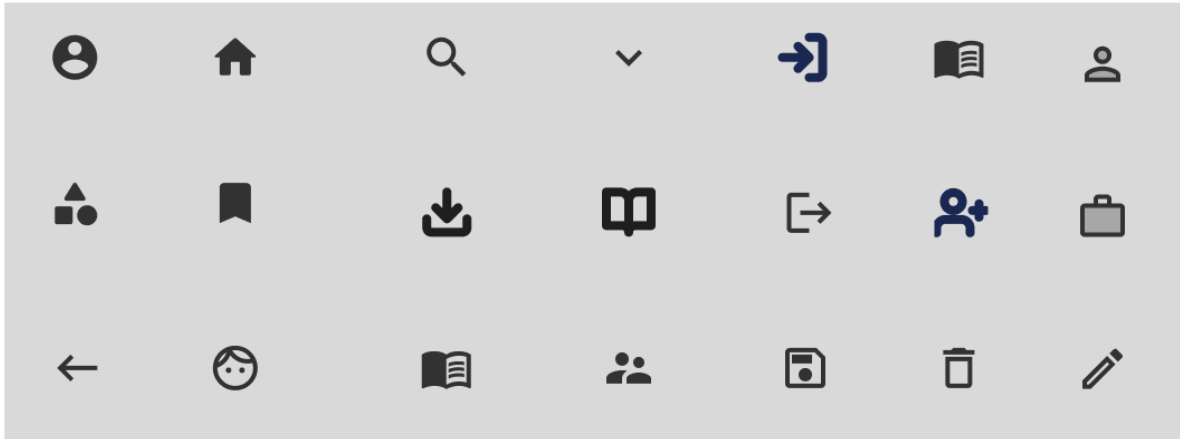
Referencia. Generados con la Herramienta de diseño "Figma"

5.4.5 Botones e iconografía

Se utilizaron íconos en formato SVG, vectorizados, obtenidos desde bancos libres de licencia como Feather Icons y Remix Icons. Los íconos se integraron al diseño del sistema con un tamaño estándar de 24x24 px, alineados con el texto.

Figura 215 Íconos y logotipos de la FGEO y del IFCP.

Iconos



Referencia. Generado con la herramienta de diseño "Figma" con la librería Material Desing Icons (Community).

Logotipo de la Fiscalía



Referencia. Obtenido de <https://fge.oaxaca.gob.mx/>

5.4.6 Componentes UI

5.4.6.1 Componente: Sidebar

La Sidebar es un componente reutilizable y adaptable que funciona como el menú lateral de navegación del sistema. Fue desarrollada para ofrecer acceso rápido y organizado a las secciones principales del sistema, manteniéndose fija en el lateral izquierdo de la pantalla.

Existen dos versiones diferenciadas por rol:

- sidebar.component.ts → para usuarios con rol **user**.
- sidebar-admin.component.ts → exclusiva para **admin** y **partner**.

Funciones principales:

- Navegación dinámica entre vistas:
 - Inicio (Home / Admin)
 - Categorías
 - Libros guardados
 - Descargas
 - Usuarios (solo admin)
 - Cerrar sesión
- Indicador visual del módulo activo, resaltando el ítem correspondiente mediante clase. `active`.
- Expansión automática al pasar el mouse (modo escritorio):
 - Reduce visualmente su tamaño (60px) cuando no está en foco.
 - Se expande (220px) al hacer hover, mostrando íconos y textos.
- Diseño responsivo, adaptándose a pantallas pequeñas.
- Permite renderizado condicional de vistas según el rol del usuario.

Aspectos técnicos:

- Estilos personalizados con CSS puro.
- Usa eventos del DOM para expansión (mouseenter y mouseleave).
- Integrada con Router de Angular para navegación directa.

Figura 219 Sidebards de rol user y admin.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.6.2 Componente: Header

El **Header** es el componente superior del sistema, visible en todas las vistas principales. Cumple la función de mostrar información contextual relevante al usuario actual.

Funciones principales:

- Mostrar el nombre del usuario autenticado.
- Mostrar la **inicial del nombre** del usuario dentro de un ícono (para identificar visualmente al perfil activo).
- Botón de **cerrar sesión**, que limpia la sesión local y redirige a la pantalla de bienvenida.
- En el caso de administradores, se puede personalizar para mostrar accesos directos o notificaciones futuras.

Características adicionales:

- Suscrito al observable currentUser\$ del AuthService para reaccionar ante cambios de estado del usuario.
- Compatible con todos los roles (admin, partner, user).
- Incluye manejo de errores en caso de que el usuario no esté disponible.

Figura 223 Header General.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.6.3 Componente: Vista previa del libro

La **vista previa del libro** es un componente implementado dentro de las secciones como "Lo nuevo", "Libros guardados" o "Libros por categoría". Su función es representar visualmente un libro individual con su miniatura, título, y acciones disponibles según el contexto.

Estructura visual:

- Imagen de portada del libro:
 - Se genera automáticamente usando pdf.js y un canvas en el frontend.
 - Muestra la primera página del PDF como **miniatura (thumbnail)**.
 - Si no está disponible, aparece el texto “No disponible”.
- Título del libro.
- Autor y editorial (en algunas vistas como “guardados” o “categoría”).
- **Acciones disponibles** (según la vista y rol):
 - Descargar: abre o guarda el archivo .pdf en el dispositivo.
 - Guardar: añade el libro al localStorage del usuario.
 -
 - Eliminar de guardados.

Lógica asociada:

- Las acciones utilizan el BookService para interactuar con el localStorage.
- Si el archivo .pdf no es accesible o la URL falla, el sistema maneja errores y muestra un fallback de imagen.

Figura 227 Modal de previsualización de libros.



Referencia. Generado con la herramienta de diseño “Figma”.

5.4.7 Interfaces de usuario

El diseño en Figma se organizó en frames separados por secciones funcionales, los cuales permitieron trabajar de manera más organizada.

5.4.7.1 Componentes más relevantes del prototipo:

5.4.7.1.1 Pantalla de Bienvenida:

Es la pantalla inicial del sistema. Funciona como portal de acceso principal y es visible tanto para usuarios nuevos como para usuarios recurrentes que aún no han iniciado sesión.

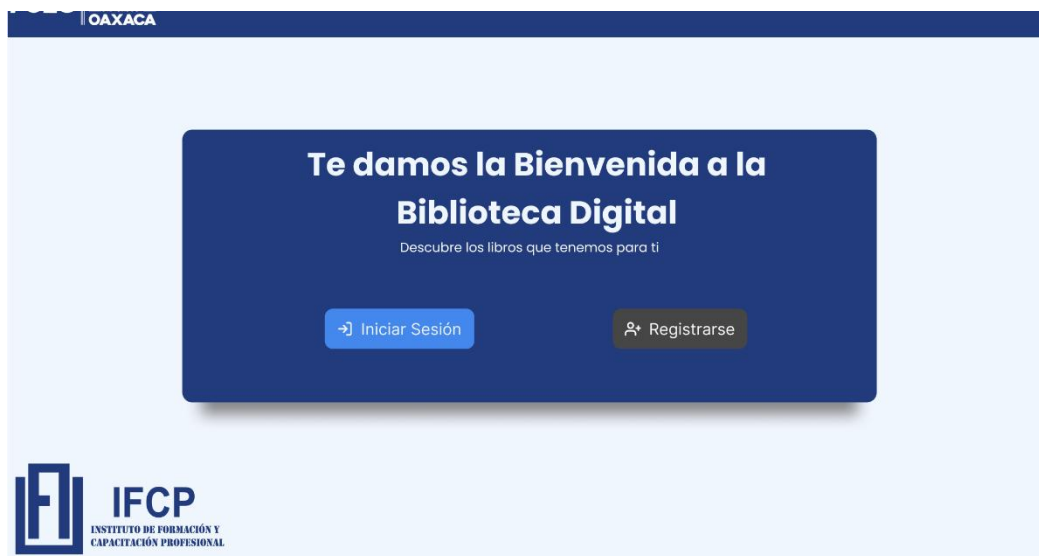
Funciones:

- Presentación visual del nombre del sistema y su propósito: una biblioteca digital.
- Botones de acción:
 - Iniciar sesión (login).
 - Registrarse (crear cuenta).
- Imagen o ícono representativo para mejorar la experiencia visual del usuario.

Características técnicas:

- Componente de tipo standalone.
- Estilo centrado, amigable y responsive.
- No requiere autenticación previa.

Figura 231 Welcome.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.7.1.2 Pantalla de login

Pantalla destinada a permitir el ingreso al sistema mediante credenciales de usuario previamente registrados. El diseño de esta vista está enfocado en la simplicidad y claridad.

Funciones:

- Ingreso de correo electrónico y contraseña.
- Validación de datos introducidos.
- Redirección automática al Dashboard correspondiente (admin/partner o user) dependiendo del rol del usuario autenticado.

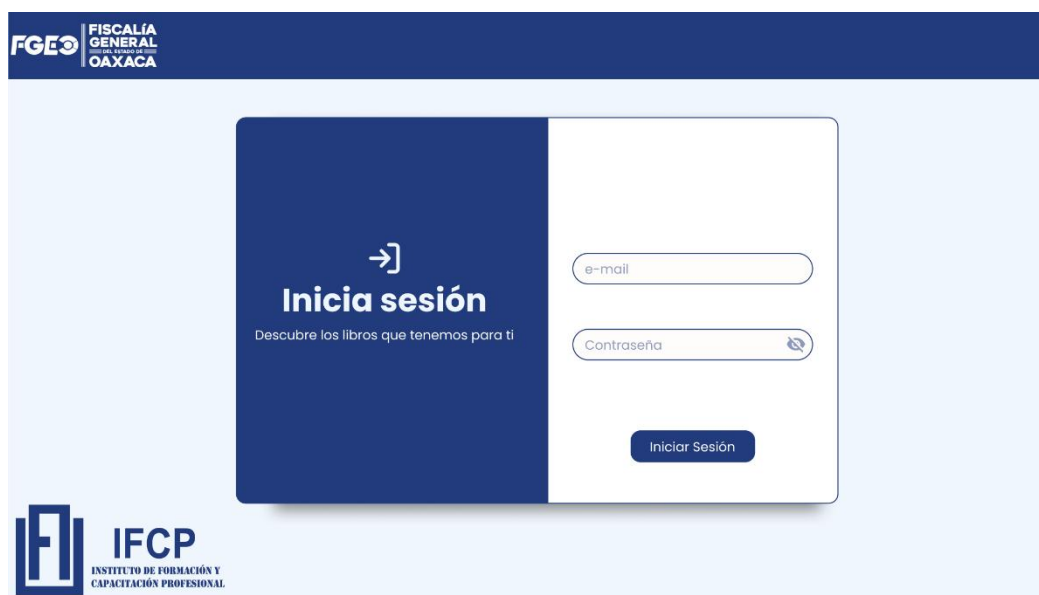
Elementos UI:

- Input de correo.
- Input de contraseña.
- Botón de acceso.
- Mensajes de error en caso de credenciales inválidas.

Seguridad:

- Comunicación segura con el backend vía token JWT.
- Almacenamiento de token y usuario autenticado en localStorage.

Figura 235 Login.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.7.1.3 Pantalla de Registro

Pantalla donde los nuevos usuarios pueden crear una cuenta en el sistema. Es una interfaz sencilla y accesible, que solicita información básica para el registro.

Campos requeridos:

- Nombre de usuario
- Nombre(s) y apellidos
- Correo electrónico
- Contraseña y confirmación
- Área de adscripción
- Puesto o rol deseado (por defecto: user)

Validaciones:

- Coincidencia de contraseñas.
- Email con formato válido.
- Todos los campos obligatorios.

Comportamiento:

- Tras completar exitosamente el registro, el usuario puede iniciar sesión desde la pantalla de login.

Figura 239 Registrar.

The image shows a registration form mockup. At the top left is the logo for FGE Oaxaca (Fiscalía General del Estado Oaxaca). The main content area has a light blue background. On the left, there is a dark grey box with a white person icon and the text 'Regístrate' and 'Se parte de nuestra historia'. To the right of this box is a white form with two sections: 'Datos personales' and 'Datos Laborales'. The 'Datos personales' section includes input fields for 'Nombres', 'Apellidos', and 'E-mail'. The 'Datos Laborales' section includes input fields for 'Puesto' and two 'Área de adscripción' fields. At the bottom left is the logo for IFCP (Instituto de Formación y Capacitación Profesional).

Referencia. Generado con la herramienta de diseño "Figma".

5.4.7.1.4 Dashboard de admin/partner

Vista principal que se muestra a los usuarios con rol admin o partner tras autenticarse. Este dashboard actúa como un centro de control, desde donde es posible gestionar libros, usuarios y categorías.

Funciones principales:

- Acceso a menú lateral con navegación hacia:
 - Gestión de libros (CRUD).
 - Gestión de categorías (CRUD).
 - Gestión de usuarios (admin exclusivamente).
 - Cierre de sesión.

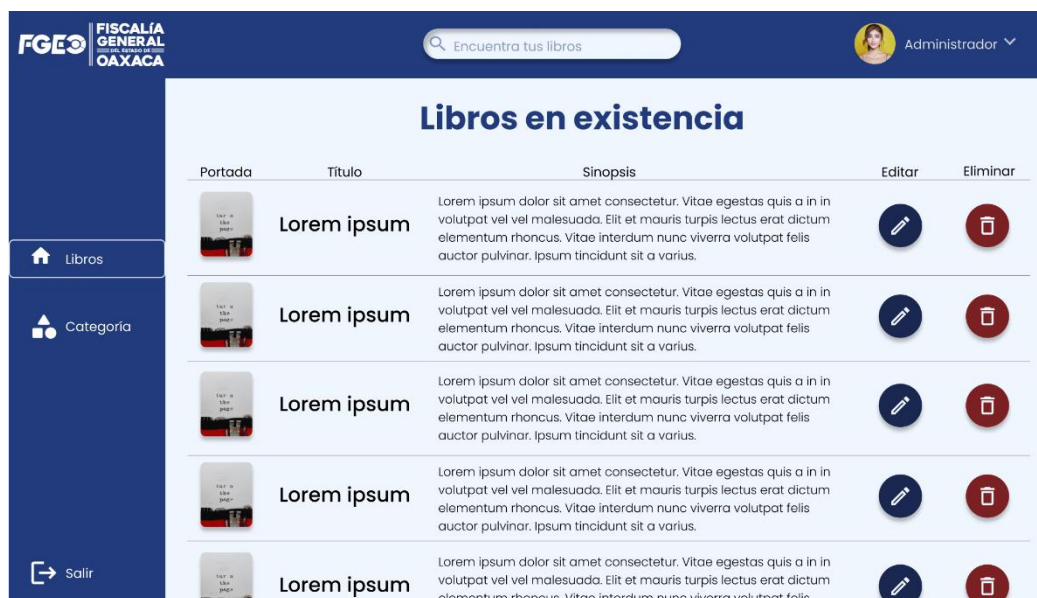
Características clave:

- Sidebar expandible al pasar el cursor.
- Indicadores visuales del apartado activo (resaltado).
- Componente de encabezado con bienvenida al usuario.
- Redirección por rol: solo admin puede asignar y revocar permisos.

Seguridad:

- Protegido por guardias (AdminGuard, RoleGuard) para impedir acceso a usuarios sin rol adecuado.

Figura 243 Home-Admin.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.7.1.5 Vista de usuarios

Esta vista está disponible exclusivamente para el usuario con rol admin. Desde aquí, el administrador puede observar todos los usuarios registrados con rol user y realizar acciones sobre ellos.

Funciones:

- Listado con tabla de usuarios (id, nombre, correo, rol, puesto, etc.).
- Botón de acción para:
 - Asignar rol partner temporalmente.
 - Revocar permisos y volver al rol user.

Diseño UI:

- Tabla responsive y clara.
- Botones con validaciones contextuales.
- Confirmaciones visuales tras acciones.

Figura 247 Home-User.



Referencia. Generado con la herramienta de diseño "Figma".

5.4.7.1.6 Vista de categorías

Vista accesible tanto para usuarios comunes como para usuarios con permisos de administración. La vista cambia ligeramente en funcionalidad según el rol del usuario.

Para usuarios (user):

- Visualización de las categorías disponibles.
- Navegación hacia los libros dentro de una categoría específica.

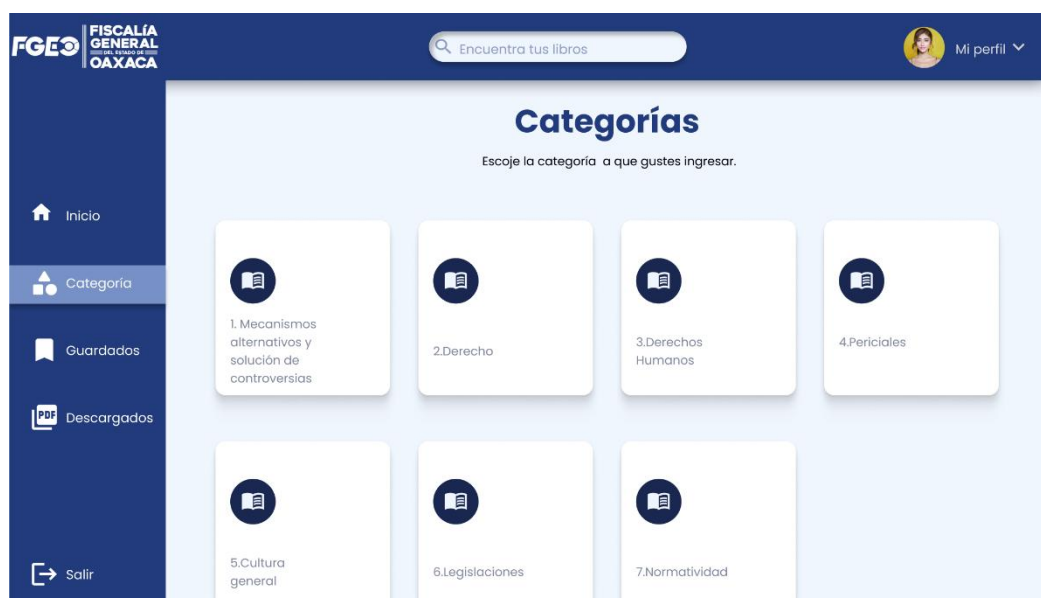
Para admin y partner:

- Además de lo anterior:
 - Agregar nueva categoría (formulario).
 - Editar nombre de categoría.
 - Eliminar categorías existentes.

Diseño funcional:

- Cards visuales por categoría.

Figura 255 Categories.



Referencia. Generado con la herramienta de diseño "Figma".

5.5 Estructura del Proyecto y Consideraciones Técnicas

5.5.1 Arquitectura Basada en Features (Feature-Based Architecture)

El proyecto fue estructurado utilizando el enfoque Feature-Based Architecture, el cual propone dividir la aplicación en módulos autónomos por funcionalidad o dominio, en lugar de agrupar archivos por tipo (por ejemplo, todos los componentes juntos, todos los servicios juntos, etc.).

Esta organización mejora la modularidad, encapsulación y cohesión del código, y permite que cada módulo crezca de forma independiente sin comprometer la estructura general.

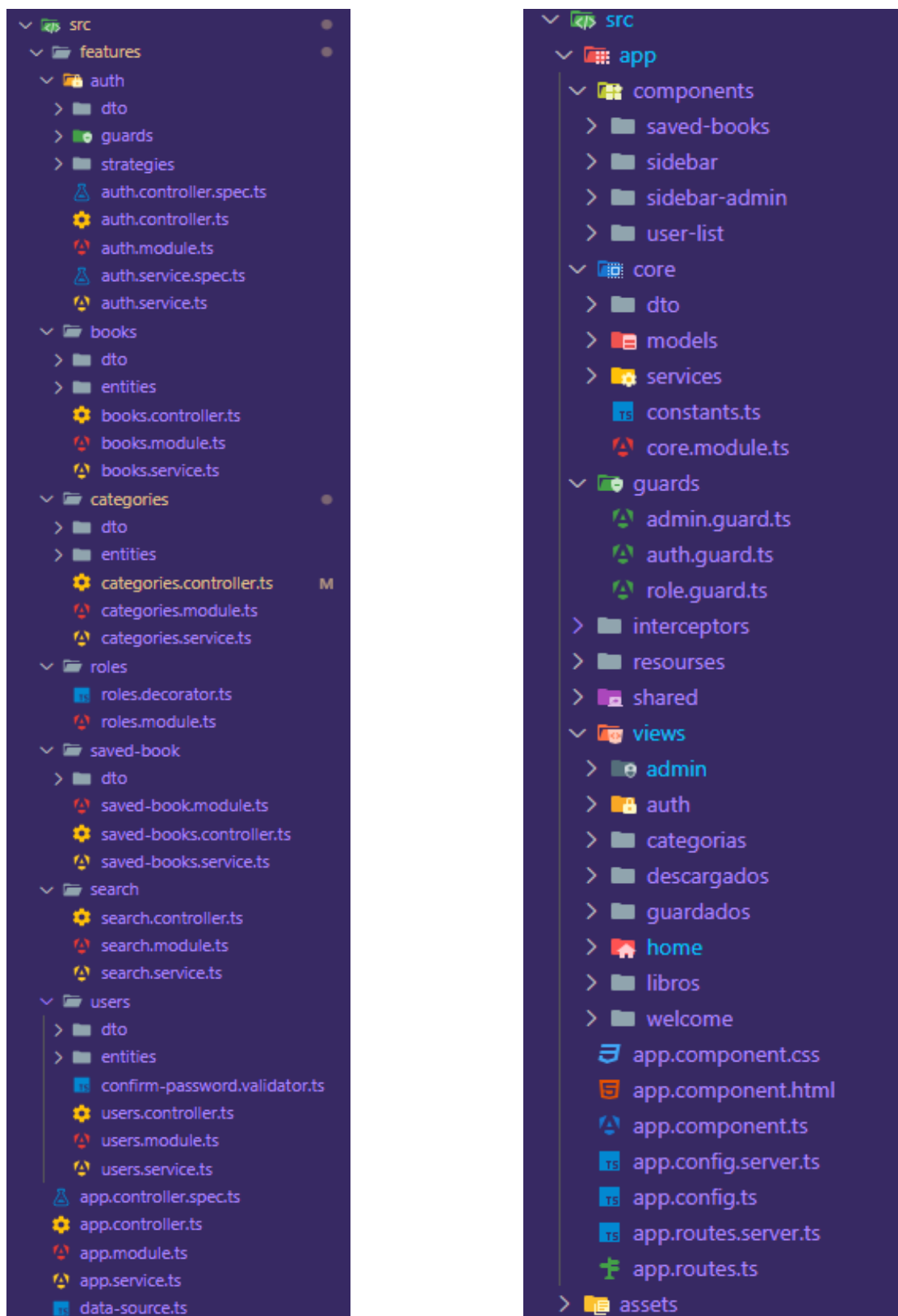
5.5.2 Estructura general del backend (NestJS) y frontend (Angular v19

standalone)

```
| -src
| ---features
| -----auth
|         | ---dto
|         | ---strategies
| -----books
|         | ---dto
|         | ---entities
| -----categories
|         | ---dto
|         | ---entities
| -----roels
| -----saved-book
|         | ---dto
| -----search
| -----users
|         | ---dto
|         | ---entities
|
| ---uploads
| -----books
|
| -.env
```

```
| -src
| ---app
| -----component
|         | ---saved-books
|         | ---sidebar
|         | ---sidebar-admin
|         | ---user-list
| -----core
|         | ---dto
|         | ---models
|         | ---services
| -----guards
| -----interceptors
| -----resources
| -----shared
| -----views
|         | ---admin
|         | ---auth
|         | ---categorias
|         | ---guardados
|         | ---home
|         | ---libros
|         | ---welcome
|
| -assets
| -enviroments
```

Figura 261 Estructura de carpetas de manera visual.



Referencia. Obtenido desde las carpetas estructuradas de "Visual Studio Code".

5.5.3 Modularidad y Reutilización

Tanto en frontend como en backend, se han definido **módulos auto contenidos** por funcionalidad, lo cual facilita:

- La **reutilización** de componentes como HeaderComponent, SidebarComponent, BookCardComponent.
- El uso de **servicios independientes** como BookService, AuthService, UserService, inyectados vía DI (Dependency Injection).
- Decoradores como @Roles() reutilizados con RolesGuard en el backend.
- Acceso compartido a configuraciones y constantes desde archivos globales.

Además, en Angular, el uso de componentes standalone y **lazy loading** de vistas permite optimizar el rendimiento, cargando solo los recursos necesarios por vista.

5.5.4 Escalabilidad y Mantenibilidad

La arquitectura del proyecto fue diseñada pensando en un **entorno productivo escalable**:

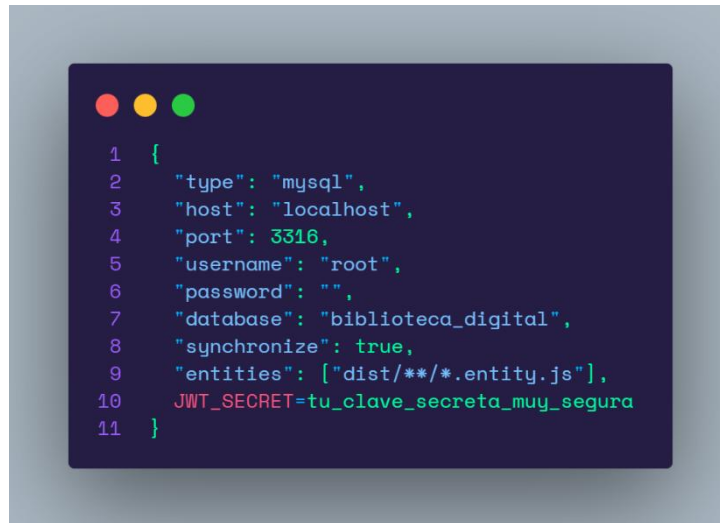
- **Soporte a múltiples roles:** admin, partner, user, con capacidad de expansión.
- **Protección por guards y decoradores:** asegurando que los endpoints y vistas están protegidos.
- **Uso de Observables y Subjects en Angular:** para flujos reactivos, actualizaciones en tiempo real y mantenibilidad.
- **Separación completa de frontend y backend:** facilita su despliegue por separado o integrado en un servidor como NGINX, Apache o contenedor Docker.

La estructura soporta escalabilidad horizontal (más servicios) y vertical (más funcionalidades) sin comprometer la integridad del sistema.

5.5.5 Variables de Entorno (.env)

El backend NestJS hace uso de un archivo .env para manejar variables sensibles y adaptables según el entorno:

Figura 265 Archivo .env.



Referencia. Código obtenido del archivo .env en el proyecto "digital_library" BackEnd .

Estas se consumen en la aplicación con @nestjs/config y permiten:

- Definir múltiples entornos: desarrollo, pruebas, producción.
- Evitar hardcodeo de credenciales.
- Reconfiguración rápida en un entorno cloud.

El frontend también puede hacer uso de variables de entorno desde **/environments/environment.ts**:

Figura 273 Archivo enviroment.ts.

5.6



Referencia. Código obtenido del archivo .env en el proyecto "Digital_library" FrontEnd.

Gestión de Versiones con Git y GitHub

La gestión de versiones es un componente esencial en el desarrollo de sistemas modernos, ya que permite mantener un control preciso sobre los cambios realizados en el código fuente, coordinar el trabajo colaborativo, revertir errores, y documentar el historial evolutivo del software. Para este proyecto, se empleó Git como sistema de control de versiones distribuido, y GitHub como repositorio remoto para alojar y administrar el código tanto del backend como del frontend.

5.6.1 Estructura de Repositorios

Se utilizaron dos repositorios independientes, uno para cada capa de la arquitectura del sistema:

5.6.1.1 Repositorio Back End (NestJS)

https://github.com/V1ctorHu/nestjs-backend_IFCP.git

5.6.1.2 Repositorio Front End (Angular v19)

https://github.com/V1ctorHu/angular-frontend_IFCP.git

5.6.2 Flujo de Trabajo con Git (Git Workflow)

- **Inicialización del proyecto:** Ambos repositorios fueron inicializados desde cero con git init, y posteriormente conectados a sus respectivos repositorios remotos en GitHub con git remote add origin.
- **Commits frecuentes y descriptivos:** Se utilizó una estrategia de commits frecuente para guardar los avances. Se empleó la convención de commits semánticos (conventional commits) para mantener un historial legible y organizado. Por ejemplo:

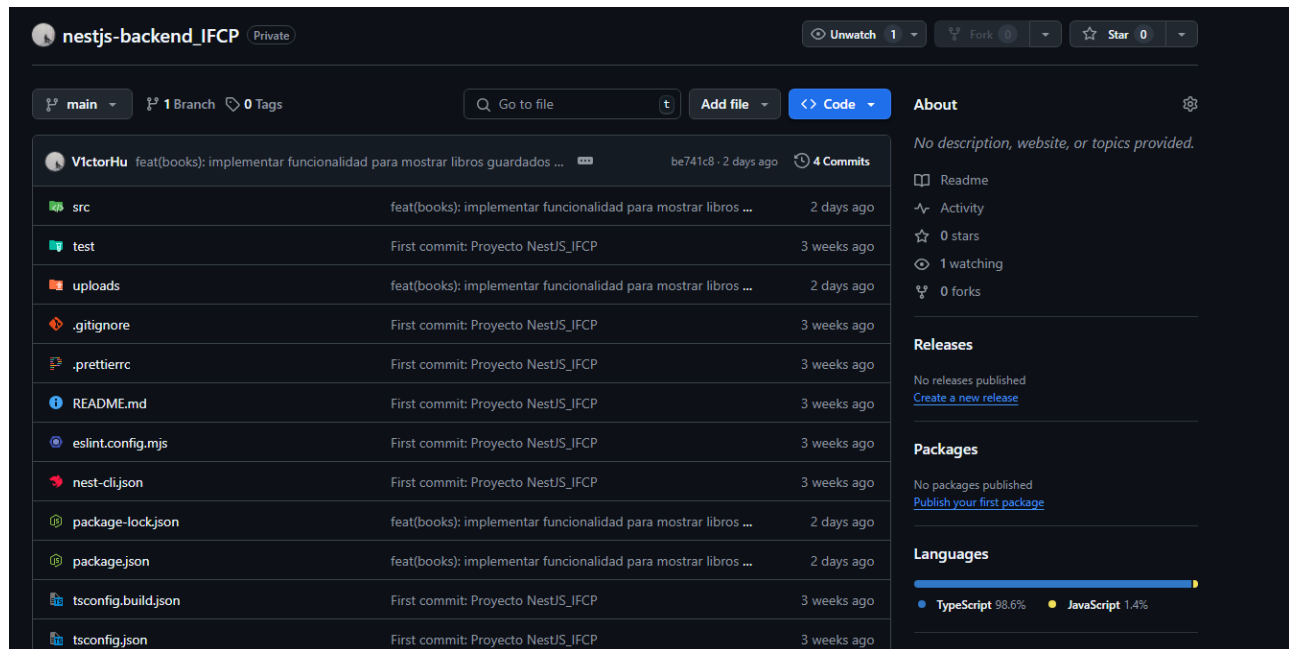
- **feat:** implementar registro de usuarios con validación
 - **fix:** corregir ruta de acceso para usuarios no autenticados
 - **refactor:** separar lógica de servicios de libros
 - **style:** mejorar diseño del formulario de login
- **Trabajo por features o secciones:** Para grandes funcionalidades, se trabajaba en ramas temporales (feature/login, feature/dashboard, etc.), que eran posteriormente integradas a la rama principal (main) mediante merge una vez validadas.
 - **Revisión y retroalimentación:** Gracias a GitHub, era posible revisar el historial de cambios, visualizar diffs, y en su caso, revertir versiones previas en caso de errores.
 - **Backup constante en la nube:** Todos los cambios eran regularmente subidos (push) a GitHub, lo que garantizaba una copia segura y accesible del código en todo momento.

5.6.3 Ventajas del Uso de Git y GitHub

- **Control total del historial:** Se puede revisar en cualquier momento cómo ha evolucionado cada archivo.
- **Trazabilidad:** Cada commit tiene asociado un mensaje descriptivo, útil para entender qué se cambió y por qué.
- **Colaboración sencilla:** GitHub permite que múltiples desarrolladores trabajen sobre el mismo proyecto sin conflictos.
- **Despliegue facilitado:** Al contar con un repositorio remoto, el proyecto puede clonarse fácilmente en servidores o entornos de staging.
- **Respaldo automático:** GitHub actúa como copia de seguridad confiable.

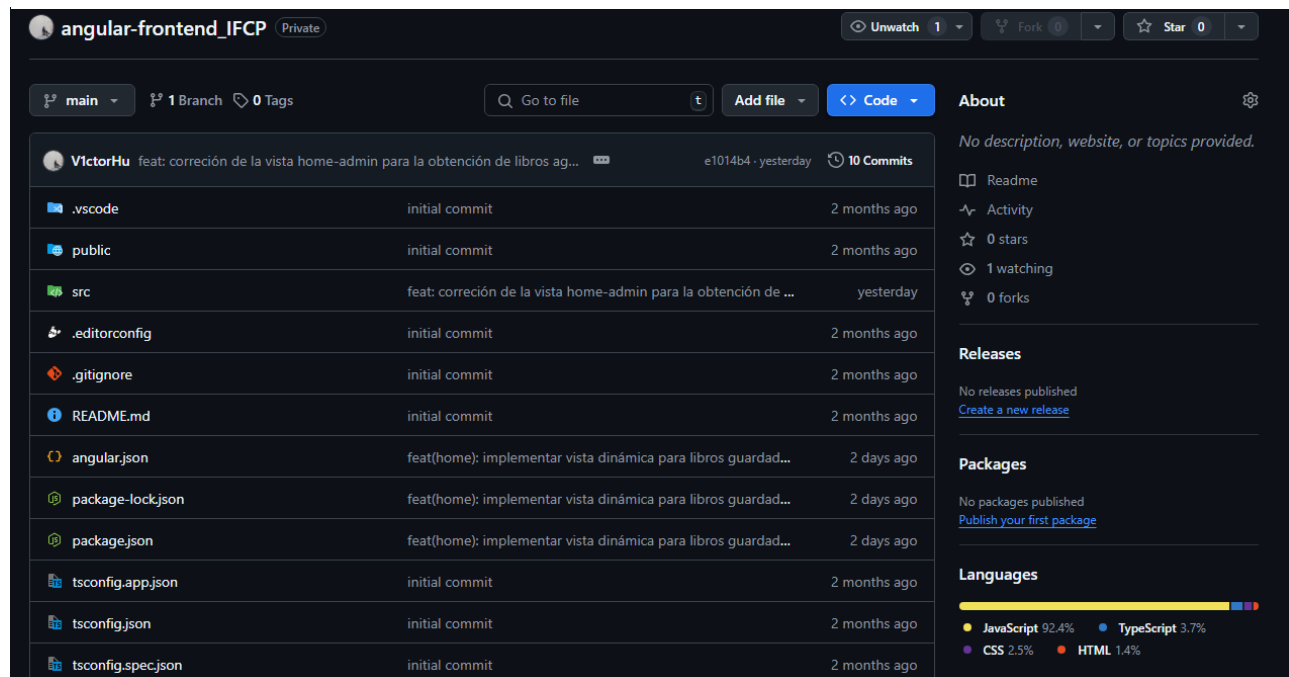
5.6.4 Estructura del repositorio Back End (NestJS) y Front End (Angular v19)

Figura 281 Repositorio en GitHub BackEnd.



Referencia. Obtenido del repositorio privado de GitHub "nestjs-backend_IFCP"

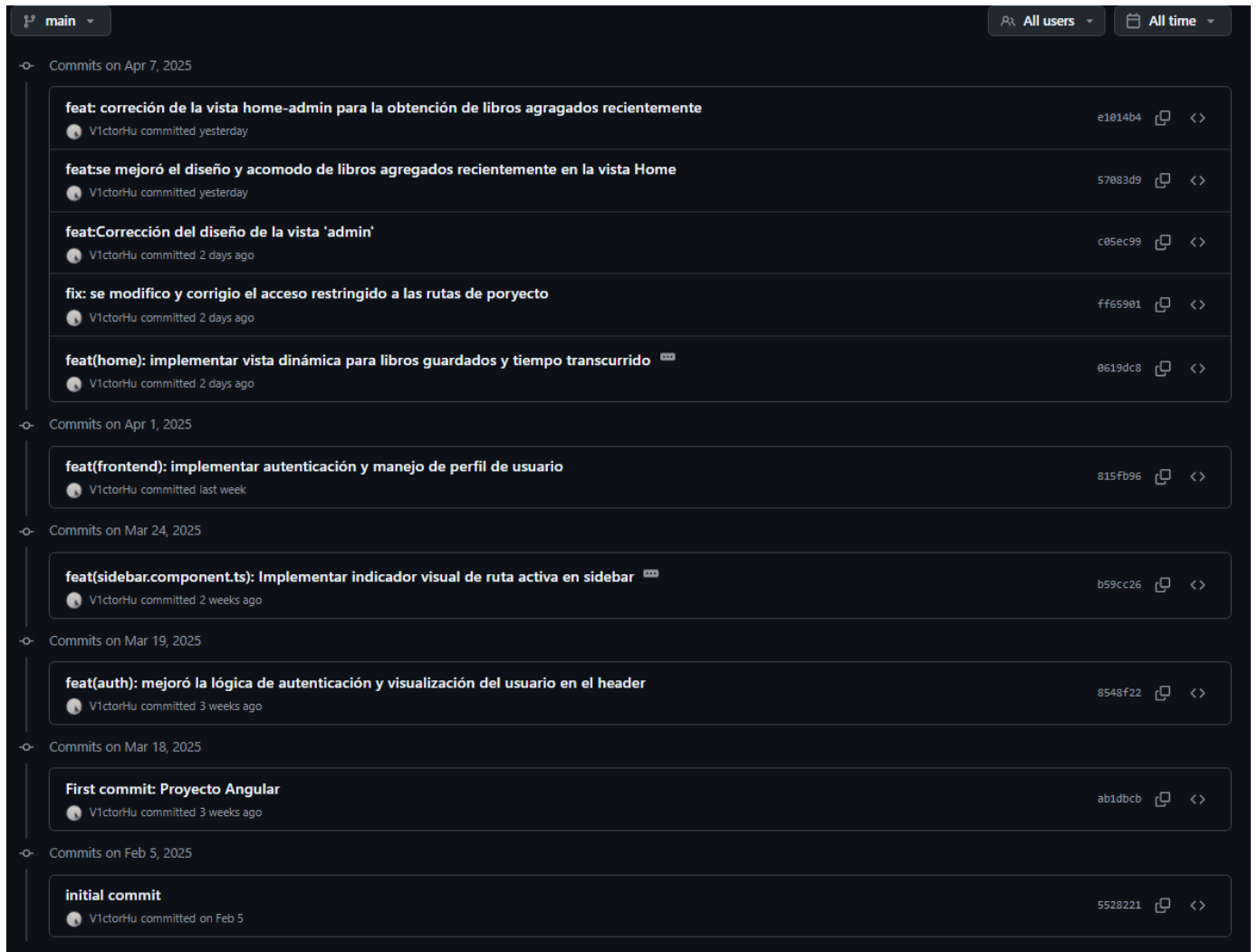
Figura 277 Repositorio en GitHub FrontEnd



Referencia. Obtenido del repositorio privado de GitHub "angular-frontend_IFCP"

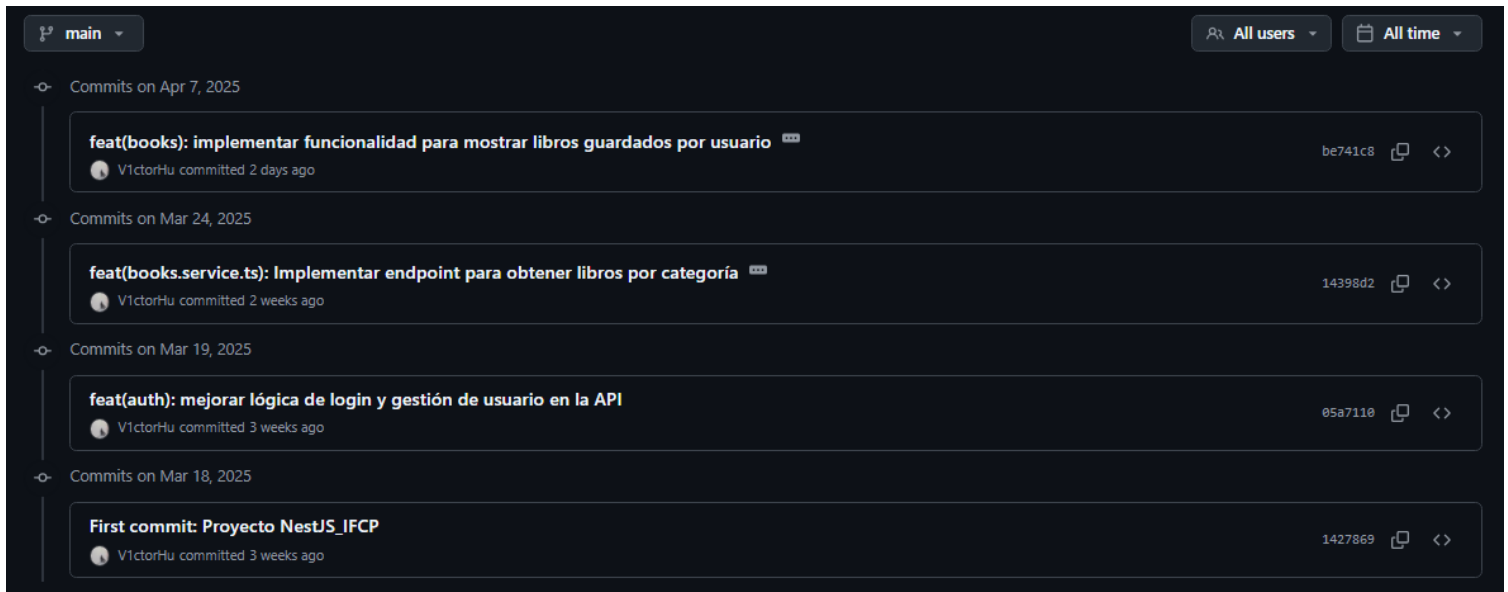
5.6.5 Ejemplo de comités realizados durante el transcurso del tiempo de desarrollo.

Figura 289 Commits totales en Frontend.



Referencia. Obtenido del repositorio privado de GitHub "angular-frontend_IFCP"

Figura 293 Commits totales en BackEnd.



Referencia. Obtenido del repositorio privado de GitHub "nestjs-backend_IFCP"

VI. RESULTADOS

A lo largo del desarrollo del sistema de biblioteca digital para el Instituto de Formación y Capacitación Profesional de la Fiscalía General del Estado de Oaxaca (IFCP), se implementaron una serie de funcionalidades que permitieron cumplir con los objetivos generales y específicos del proyecto. A continuación, se detallan los resultados obtenidos:

- Interfaz de usuario amigable y funcional, desarrollada con Angular v19 bajo una arquitectura Feature-Based Standalone, lo cual permitió una navegación clara e intuitiva para todos los perfiles de usuario (admin, partner y usuario general).
- Back-end modular en NestJS, utilizando controladores, servicios, repositorios y middleware (como guards y decorators) para el control de roles, seguridad de acceso y autenticación por token JWT. Se validó la escalabilidad de los módulos y su mantenimiento en producción.
- Sistema de roles efectivo, con restricciones aplicadas mediante guards que permiten que sólo usuarios con roles admin o partner puedan realizar acciones críticas como editar, eliminar, crear o asignar permisos.
- Funcionalidad CRUD completa para los módulos de libros, categorías y usuarios, incluyendo manejo de archivos (subida y descarga de PDFs), visualización de miniaturas con PDF.js, y guardado local de libros.
- Base de datos estructurada y optimizada en MariaDB con relaciones entre libros, usuarios, categorías y libros guardados (saved_books). Se respetaron las buenas prácticas de integridad referencial mediante llaves foráneas.
- Integración visual con Figma, aplicando una guía de estilo definida: paleta de colores institucionales, uso de fuentes legibles (Poppins), botones consistentes e inputs estandarizados.
- Implementación del guardado local (localStorage), que permitió simular una experiencia de persistencia para los usuarios sin necesidad de una base de datos externa en esta fase del proyecto.

- Uso de Git y GitHub para la gestión de versiones y control de cambios, dividiendo adecuadamente el repositorio del frontend y backend. Se aplicó la convención convencional commits para registrar avances.
- Simulación de despliegue con configuración de variables `.env`, rutas absolutas y optimización de assets para su futura subida a un servidor en producción.

VII. CONCLUSIONES

Con base en los resultados obtenidos, la hipótesis planteada fue comprobada satisfactoriamente. La creación de una biblioteca digital centralizada permitió optimizar significativamente la gestión, búsqueda y distribución de recursos bibliográficos dentro del Instituto de Formación y Capacitación Profesional (IFCP).

Se logró demostrar que la implementación de una solución digital moderna y segura mejora no solo la accesibilidad y la preservación del conocimiento, sino también el flujo organizativo interno del personal, facilitando la capacitación y el acceso a material académico.

Asimismo, durante el desarrollo surgieron aspectos no previstos inicialmente, como:

- La necesidad de implementar una vista exclusiva para usuarios con rol partner que no estuviera contemplada desde el inicio.
- La relevancia de permitir el guardado de libros de forma local por parte de los usuarios finales (feature que mejoró sustancialmente la experiencia de uso).
- El reto técnico de generar miniaturas desde archivos PDF sin sobrecargar el backend, resuelto exitosamente en el frontend con PDF.js.

El sistema desarrollado es aplicable a cualquier institución que busque digitalizar su biblioteca o recursos internos. Gracias a su diseño modular, puede escalarse fácilmente, añadiendo nuevas funcionalidades como comentarios, calificaciones o integración con sistemas bibliotecarios internacionales.

En conclusión, este proyecto no solo resolvió una problemática institucional, sino que también sentó las bases para una transformación digital completa, replicable y sostenible dentro del ámbito gubernamental y educativo.

BIBLIOGRAFÍA

- Open Library*. (s.f.). Obtenido de <https://openlibrary.org/>
- Admltsqmet. (12 de Diciembre de 2022). *INSTITUTO TECNOLÓGICO UNIVERSITARIO QUITO METROPOLITANO*. Obtenido de <https://itsqmet.edu.ec/los-8-tipos-de-metodologia-de-desarrollo-de-software/#:~:text=%C2%BFQu%C3%A9%20es%20una%20metodolog%C3%ADa%20de,decisi%C3%B3n%20del%20equipo%20cual%20implementarla>.
- Angular. (2023). *Angular.dev*. Obtenido de <https://angular.io/>
- Angular. (2025). *Angular Material*. Obtenido de Material Design components for Angular: <https://material.angular.io/>
- Atlassian. (2024). *What is version control?* Obtenido de <https://www.atlassian.com/git/tutorials/what-is-version-control>
- Atlassian. (s.f.). *Scrumban: domina dos metodologías ágiles*. Obtenido de <https://www.atlassian.com/es/agile/project-management/scrumban>
- Auth0. (2023). *JSON Web Tokens*. Obtenido de <https://auth0.com/docs/secure/tokens/json-web-tokens#benefits-of-jwts>
- Auth0. (2023). *JWT (JSON Web Tokens) introduction*. Obtenido de <https://auth0.com/docs/secure/tokens/json-web-tokens>
- Auth0. (2023). *What is JSON Web Token?* Obtenido de <https://auth0.com/learn/json-web-tokens>
- Chen, P. P.-S. (1 de 03 de 1976). *The entity-relationship model—toward a unified view of data*. Obtenido de <https://dl.acm.org/doi/10.1145/320434.320440>
- Elmasri, R. (2016). *Fundamentals of database systems* (7th ed.). <https://www.pearson.com>.
- Encrypt, L. (2023). *NestJS framework overview*. Obtenido de <https://docs.nestjs.com>
- Encrypt, L. (2023). *What is HTTPS?* Obtenido de <https://letsencrypt.org/>
- Fielding, R. T. (2000). *Architectural Styles and the Desing of Network-based Software Architectures*.
- Figma.com. (2025). *Figma*. Obtenido de <https://www.figma.com/>
- Foundation, M. (2009 - 2025). *MariaDB Server: the innovative open source database*. Obtenido de <https://mariadb.org/>
- Git. (2025). *Git --fast-version-control*. Obtenido de <https://git-scm.com/>
- GitHub. (2024). *GitHub*. Obtenido de <https://github.com/>
- HeidiSQL. (2025). *What is HeidiSQL?* Obtenido de <https://www.heidisql.com/>
- Hernández Sampieri, R. F. (2014). *Metodología de la investigación* (6ª ed.). México: McGraw-Hill.
- Library Genesis*. (s.f.). Obtenido de <https://libgen.is/>

Marcotte, E. (2011). *Responsive Web Design*. Obtenido de <https://abookapart.com/products/responsive-web-design#quote-3>

MariaDB. (2025). *MariaDB Server: the innovative open source database*. Obtenido de <https://mariadb.org/>

Material, A. (2023). *Angular Material documentation*. Obtenido de <https://material.angular.io>

MDN. (2025). *Acerca de JavaScript*. Obtenido de <https://developer.mozilla.org/es/docs/conflicting/Web/JavaScript>

Microsoft. (13 de 06 de 2023). *Uso de un modelo de arquitectura de Three-Tier*. Obtenido de <https://learn.microsoft.com/es-es/windows/win32/cos-sdk/using-a-three-tier-architecture-model>

NestJS. (2025). *Hello, nest!* Obtenido de <https://nestjs.com/>

NestJS, D. (2023). *Introduction to NestJS*. Obtenido de <https://docs.nestjs.com/>

NodeJS. (2025). *Ejecuta JavaScript en cualquier parte*. Obtenido de <https://nodejs.org/es>

Norman, D. (10 de 01 de 2014). *The Design of Everyday Things*. Obtenido de <https://mitpress.mit.edu/9780262525671/the-design-of-everyday-things/>

npmjs. (2025). *Build amazing things*. Obtenido de <https://www.npmjs.com/>

OWASP. (2023). *Cross-Site Scripting (XSS) prevention cheat sheet*. Obtenido de <https://owasp.org/www-project-cheat-sheets/>

Peño, J. M. (2015). *Pruebas de Software. Fundamentos y Técnicas*.

Postman. (2025). *Post API Platform*. Obtenido de <https://www.postman.com/>

Ramez Elmasri, S. B. (2016). *Pearson., Fundamentals of Database Systems (7th ed.)*.

Rouse, M. (9 de Abril de 2021). *ComputerWeekly.es*. Obtenido de https://www.computerweekly.com/es/definicion/Base-de-datos-relacional?_gl=1*1h2p1b1*_ga*NzUxMTc1MDgwLjE2ODM3MzI3NDY.*_ga_TQKE4GS5P9*MTY4MzczMjc0Ni4xLjEuMTY4MzczMzI4NC4wLjAuMA..*_ga_RZDF13FDNT*MTY4MzczMjc0Ni4xLjEuMTY4MzczMzI4NC4wLjAuMA..*_ga_NLDTRJGG3Y*MTY

Services), A. (. (2024). *¿Qué es SQL (lenguaje de consulta estructurada)?* Obtenido de [https://aws.amazon.com/what-is/sql/#:~:text=Structured%20query%20language%20\(SQL\)%20is,relationships%20between%20the%20data%20values](https://aws.amazon.com/what-is/sql/#:~:text=Structured%20query%20language%20(SQL)%20is,relationships%20between%20the%20data%20values).

Sotomayor, S. G. (9 de 11 de 2024). *iebschool*. Obtenido de Metodologías ágiles: ¿Qué son y cuáles son más utilizadas?: <https://www.iebschool.com/blog/que-son-metodologias-agiles-agile-scrum/>

Studio, V. (2025). <https://code.visualstudio.com/>. Obtenido de <https://code.visualstudio.com/>

Tailwind. (2025). *Rapidly build modern websites without ever leaving your HTML*. Obtenido de <https://tailwindcss.com/>

Team, A. (2023). *Angular documentation*. Obtenido de <https://angular.io/docs>

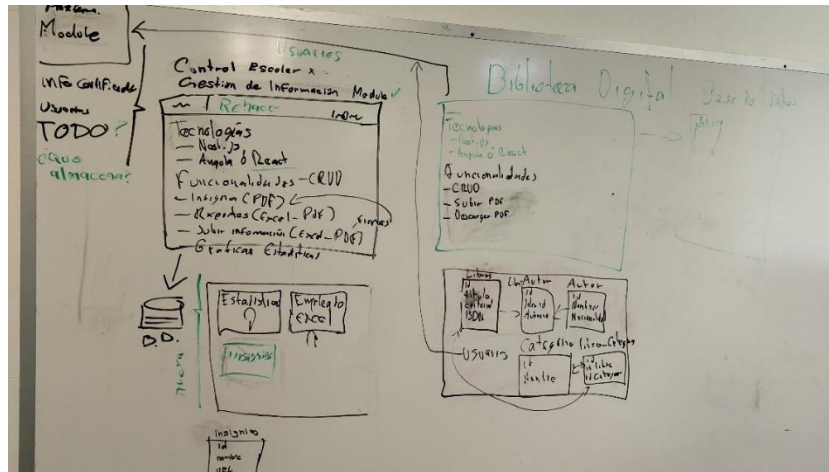
TypeScript. (2025). *TypeScript is JavaScript with syntax for types*. Obtenido de <https://www.typescriptlang.org/>

W3C, W. A. (2023). *Web Content Accessibility Guidelines (WCAG)*. Obtenido de <https://www.w3.org/WAI/standards-guidelines/wcag/>

WHATWG. (8 de 03 de 2025). *HTML*. Obtenido de Living Standard : <https://html.spec.whatwg.org/>

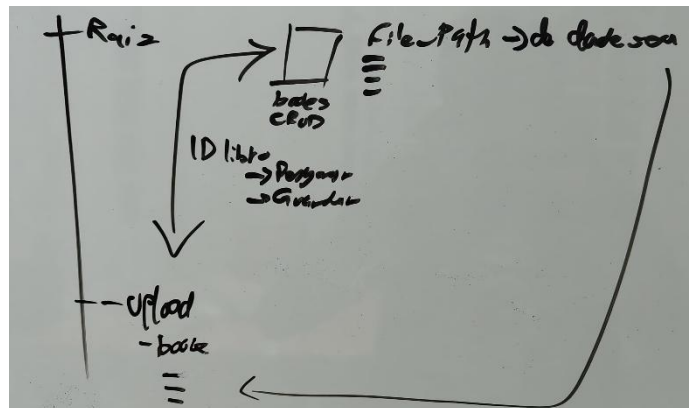
ANEXOS

Anexos 1 Boceto de la creación de tablas.



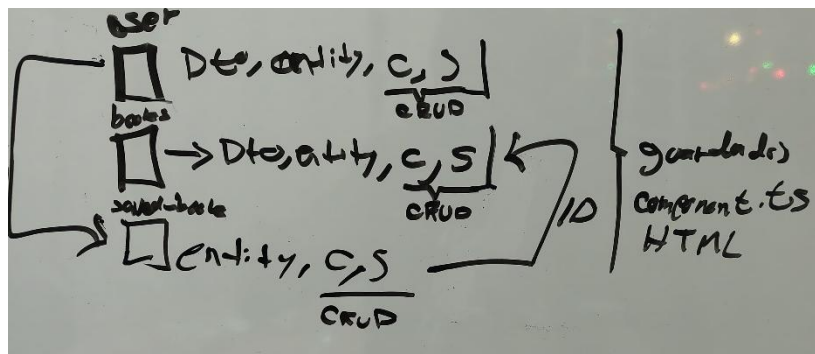
Referencia. Fotografía tomada desde el Smartphone Nothing Phone (2).

Anexos 3 Boceto de la función Uploads_books.



Referencia. Fotografía tomada desde el Smartphone Nothing Phone (2).

Anexos 2 Boceto de la función Saved_book por id de usuario.



Referencia. Fotografía tomada desde el Smartphone Nothing Phone (2).